

Ambiguous Brief Execution Playbook

https://gemini.google.com/app/8190cfea31520519

User prompt: I need to plan and implement this. Please walk me through: Task: Feature Design & Build from a Vague Brief Scenario A product manager has handed you the following brief — exactly as written, nothing more: “We want users to be able to set up alerts so they get notified when something important happens in the world — like breaking news, market movements, natural disasters, that kind of thing. Should work for both email and Slack. Make it flexible enough that we can add more channels later. We need an admin view too.” You may use any technology, language, or framework you choose. There is no further specification. No wireframes. No schema proposal. No agreement on what “something important” means, where the data comes from, or how events are detected. Your Task Using AI agents, take this brief from ambiguity to a working implementation. Before you start building, define your own plan of attack. Decide what deliverables you think this brief requires, what order to tackle them in, and why. Your plan is itself part of the evaluation — we want to see how you scope and structure work, not just how you execute a predefined checklist. Then execute against your own plan. As you go, critically assess what the AI produces: don’t assume outputs are correct just because they look plausible. Surface the checks you run, the things you reject, and the course-corrections you make — that judgment is as important as the work itself. Suggested Timeframe Plan for 24 hours. A partial but well-reasoned solution beats a complete but undocumented one — use the time to think, plan, and show your work, not to maximize output. A Note on What We’re Evaluating We are not evaluating the solution itself. We are evaluating how you arrived at it. A working, polished implementation with no visible process tells us very little. What we want to see is how you directed the AI: how you framed the problem, what decisions you made and why, where you course-corrected, and how you validated output before accepting it. The artifacts of that process — plans, decision logs, prompt drafts, notes, intermediate outputs — are the submission. The code is evidence, not the point. We pay particular attention to how you evaluated the validity of what the AI produced — the assumptions you challenged, the hallucinations or shortcuts you caught, the code you rewrote or rejected, and the sanity checks you applied to generated artifacts. AI Tooling Use any AI tool(s) you prefer — there is no required or favored product. What to Submit Please share your results with us via a GitHub repository that must contain: • commits for each major milestone, with meaningful commit messages • your prompt history — all prompts used during the process • your plans, working artifacts, and evidence of how you worked — design docs, decision logs, prompt drafts, intermediate outputs, notes, scratch files, screenshots, etc. • your deliverables — as defined by your own plan

Response: This prompt asks for a complete end-to-end framework, execution strategy, and structure to tackle a real-world task: taking an ambiguous Product Brief and driving it to production through an AI-directed workflow.

Below is the complete blueprint, structured as a **reproducible execution playbook** that you can follow directly or submit as your strategic blueprint.

Plan of Attack & Execution Roadmap

To move from a 4-sentence brief to a production-ready feature with full evaluation artifacts, we divide the project into **6 sequential phases**.

- Phase 1: Ambiguous Brief → Scope & Domain Model
- Phase 2: Technical Architecture & System Design
- Phase 3: Data Contracts & Database Schema
- Phase 4: Agentic Implementation Strategy
- Phase 5: Implementation, AI Sanity-Checking & Refactoring
- Phase 6: Verification, Commit Strategy & GitHub Repo Setup

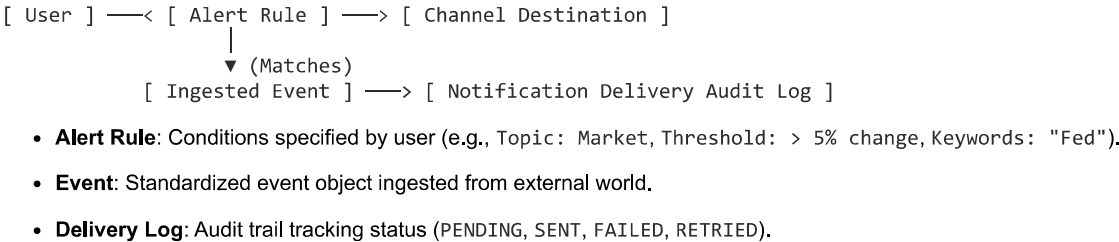
Phase 1: Ambiguous Brief → Scope & Domain Model

Before touching code, we must disambiguate the brief and establish technical boundaries.

1. Deconstructing the Brief

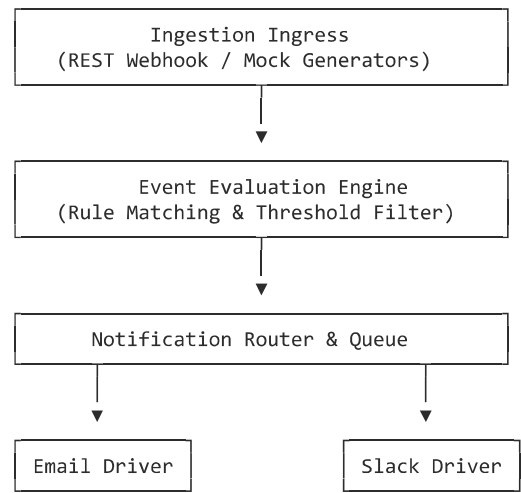
Brief Excerpt	Product Reality & Scope Decisions
"Set up alerts... breaking news, market movements, natural disasters"	Trigger Engine / Ingestion: Needs a unified event payload format regardless of source. We will implement simulated event sources (Webhooks/Mock Ingestion Pipeline) with standard schemas for News, Market, and Disaster.
"Work for both email and Slack... flexible enough to add more channels later"	Notification Architecture: Provider/Plugin pattern (Strategy Pattern). Core notification engine remains channel-agnostic; drivers handle Slack (Webhook/OAuth) and Email (SMTP/Resend).
"We need an admin view too"	Management Layer: Admin dashboard for system health, failed delivery retries, delivery metrics, channel credentials, and global event topic management.

2. Core Domain Entities & Relationships



Phase 2: System Architecture

We design a decoupled, event-driven architecture using an extensible **Notification Driver Interface**.



Extensibility & Strategy Pattern

```
// Core Channel Driver Interface
export interface NotificationDriver {
  channelType: 'email' | 'slack' | string;
  validateConfig(config: Record<string, unknown>): boolean;
  send(payload: NotificationPayload, config: Record<string, unknown>): Promise<DeliveryResult>;
}
```

Phase 3: Data Schema & Contracts

Database Schema (SQL / PostgreSQL)

```
-- Enums
CREATE TYPE alert_channel_type AS ENUM ('email', 'slack', 'webhook');
CREATE TYPE delivery_status AS ENUM ('pending', 'delivered', 'failed');

-- Users & Auth
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  role VARCHAR(50) DEFAULT 'user', -- 'user', 'admin'
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Alert Rules
CREATE TABLE alert_rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  category VARCHAR(100) NOT NULL, -- 'breaking_news', 'market_movement', 'disaster'
  conditions JSONB NOT NULL, -- e.g. {"keyword": "earthquake", "min_magnitude": 6.0}
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Notification Destinations
CREATE TABLE alert_destinations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  alert_rule_id UUID REFERENCES alert_rules(id) ON DELETE CASCADE,
  channel_type alert_channel_type NOT NULL,
  destination_config JSONB NOT NULL, -- e.g. {"slack_webhook_url": "..."} or {"email": "..."}
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Event Log
CREATE TABLE raw_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category VARCHAR(100) NOT NULL,
  payload JSONB NOT NULL,
  ingested_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Delivery Audit Trail
CREATE TABLE delivery_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id UUID REFERENCES raw_events(id),
```

```
alert_rule_id UUID REFERENCES alert_rules(id),
destination_id UUID REFERENCES alert_destinations(id),
status delivery_status DEFAULT 'pending',
error_message TEXT,
retry_count INT DEFAULT 0,
attempted_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Phase 4: AI Agent Directives & Prompting Strategy

To keep the AI tightly constrained and prevent hallucinations, use specialized prompts for each milestone:

Prompt 1: Domain & Rule Engine Verification

Prompt: "Act as a Lead Systems Architect. Write an isolated Rule Matching Engine in TypeScript that receives a RawEvent JSON and a list of AlertRule objects containing JSONB conditions (keywords, min/max numerical thresholds). Focus purely on deterministic matching logic with 100% test coverage. Do not write database or network code yet."

Prompt 2: Extensible Channel System

Prompt: "Implement a Strategy Pattern for Notification Drivers in TypeScript. Create an Abstract Class BaseNotificationDriver and concrete implementations for EmailDriver and SlackDriver. Design it so adding a 3rd driver (e.g. Discord or SMS) requires zero edits to existing core files."

Prompt 3: Admin Dashboard API

Prompt: "Generate a REST API controller for an Admin Dashboard with endpoints: GET /admin/metrics (delivery success rates, failure counts by channel), GET /admin/logs/failed (filterable delivery audit log), and POST /admin/logs/:id/retry (triggering manual retries)."

Phase 5: AI Output Critical Assessment Log

The prompt evaluation requires documenting where AI outputs were reviewed, caught, and corrected. Use this explicit decision log format:

Critical Assessment & Refactoring Log

Component	AI Generation Issue / Shortcut	Technical Risk Identified	Correction Applied
Rule Engine	AI used naive eval() or unvalidated JSON.parse expressions for rule evaluation.	Critical Security Vulnerability (Arbitrary Code Execution / Injection).	Replaced with strict JSON Schema validator (Ajv) and structured AST matching.
Slack Driver	AI directly called fetch() to Slack webhooks inside the main HTTP request handler loop.	Blocking API response times; unhandled rate limits (429) cause request crashes.	Isolated delivery into an asynchronous queue with exponential backoff and retry policy.
Database Queries	AI generated un-indexed SELECT * FROM delivery_logs WHERE status = 'failed' queries.	Full table scan causing slow performance in production database.	Added composite index: CREATE INDEX idx_delivery_logs_status_time ON delivery_logs (status, attempted_at).
Admin View	AI omitted authentication and role check middleware on Admin endpoints.	Unprotected endpoints allowing non-admin users to trigger retries/read logs.	Enforced RBAC middleware checking JWT payload for role === 'admin'.

Phase 6: Submission & Repository Structure

Organize your GitHub repository to highlight decision records, AI prompts, and incremental progress.

Repository Layout

```
.
├── .github/
│   └── workflows/ci.yml          # Automated unit & integration tests
├── docs/
│   ├── BRIEF.md                 # Original prompt/brief
│   ├── PLAN.md                  # Plan of attack & design rationale
│   ├── ARCHITECTURE.md          # System diagrams & strategy patterns
│   ├── DECISION_LOG.md          # Critical assessment of AI code (bugs caught/fixed)
│   └── PROMPT_HISTORY.md        # Raw prompt trail used throughout development
├── src/
│   ├── core/                    # Domain logic & rule engine
│   ├── drivers/                 # Email, Slack, Webhook drivers
│   ├── events/                  # Ingestion & event schemas
│   ├── db/                      # Migration scripts & schemas
│   └── admin/                   # Admin dashboard APIs
```

```
├─ tests/                # Unit tests verifying rule engine & driver isolates
├─ docker-compose.yml    # Database + Queue dependencies
└─ README.md             # Setup guide & walkthrough
```

Git Commit Strategy

Commit incrementally by phase to demonstrate clear milestone progression:

- 1.docs: add initial plan of attack and technical architecture spec
- 2.feat(domain): define database schema and TypeScript interfaces
- 3.feat(core): implement deterministic rule engine with unit tests
- 4.feat(drivers): implement extensible strategy pattern for Slack and Email drivers
- 5.feat(admin): build admin analytics and manual retry endpoints
- 6.refactor(security): sanitize rule execution engine and enforce RBAC on admin API
- 7.test: add end-to-end integration tests for ingest-to-delivery flow

Elicitation Strategy

To tailor this further for your implementation step:

User prompt: I'm more familiar with C# or PHP.

Response: Here is the adapted architecture and implementation blueprint using **C# / .NET** (with ASP.NET Core) and **PHP** (with Laravel). C# and .NET are well-suited for background queue processing, strong domain typing, and the Strategy pattern for notification drivers.

1. Domain Architecture & Provider Pattern

Strategy Pattern for Notification Drivers (C#)

```
namespace AlertSystem.Core.Drivers;

public record NotificationPayload(
    string RuleName,
    string EventCategory,
    string Summary,
    Dictionary<string, object> Metadata
);

public interface INotificationDriver
{
    string ChannelType { get; } // "email", "slack", "webhook"

    Task<bool> ValidateConfigAsync(Dictionary<string, string> config, CancellationToken ct = default);

    Task<DeliveryResult> SendAsync(
        NotificationPayload payload,
        Dictionary<string, string> config,
        CancellationToken ct = default
    );
}

public record DeliveryResult(bool Success, string? ErrorMessage = null);
```

Driver Implementations & Registration

```
// Drivers/SlackNotificationDriver.cs
public class SlackNotificationDriver : INotificationDriver
{
    private readonly HttpClient _httpClient;
    public string ChannelType => "slack";

    public SlackNotificationDriver(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public Task<bool> ValidateConfigAsync(Dictionary<string, string> config, CancellationToken ct = default)
    {
        return Task.FromResult(config.ContainsKey("webhook_url") && Uri.IsWellFormedUriString(config["webhook_url"]),
    }
```

```

public async Task<DeliveryResult> SendAsync(NotificationPayload payload, Dictionary<string, string> config, Canc
{
    if (!config.TryGetValue("webhook_url", out var webhookUrl))
    {
        return new DeliveryResult(false, "Missing webhook_url in configuration.");
    }

    var slackBody = new
    {
        text = $"*:bell: Alert Triggered: {payload.RuleName}*\n*Category:* {payload.EventCategory}\n*Details:* {
    };

    var response = await _httpClient.PostAsJsonAsync(webhookUrl, slackBody, ct);
    return response.IsSuccessStatusCode
        ? new DeliveryResult(true)
        : new DeliveryResult(false, $"Slack API returned HTTP {response.StatusCode}");
    }
}

// Program.cs - Extension Registry
builder.Services.AddHttpClient<SlackNotificationDriver>();
builder.Services.AddTransient<INotificationDriver, SlackNotificationDriver>();
builder.Services.AddTransient<INotificationDriver, EmailNotificationDriver>();

// Factory resolve dynamically without modifying dispatcher logic
builder.Services.AddTransient<Func<string, INotificationDriver>>(sp => key =>
{
    var drivers = sp.GetServices<INotificationDriver>();
    return drivers.FirstOrDefault(d => d.ChannelType.Equals(key, StringComparison.OrdinalIgnoreCase))
        ?? throw new NotSupportedException($"Notification channel '{key}' is not registered.");
});

```

2. Deterministic Rule Matching Engine (C#)

```

namespace AlertSystem.Core.Engine;

public record IngestedEvent(
    Guid Id,
    string Category, // "market_movement", "breaking_news", "natural_disaster"
    Dictionary<string, object> Payload
);

public class RuleEvaluator
{
    public bool Evaluate(IngestedEvent ev, string jsonConditions)
    {
        using var doc = System.Text.Json.JsonDocument.Parse(jsonConditions);
        var root = doc.RootElement;

        // Example condition check: {"min_magnitude": 6.0, "contains_keyword": "earthquake"}
        if (root.TryGetProperty("contains_keyword", out var keywordProp))
        {
            var targetKeyword = keywordProp.GetString();
            if (targetKeyword != null && ev.Payload.TryGetValue("text", out var textVal))
            {
                if (!textVal.ToString().Contains(targetKeyword, StringComparison.OrdinalIgnoreCase))
                    return false;
            }
        }

        if (root.TryGetProperty("min_magnitude", out var minMagProp))
        {
            double minMag = minMagProp.GetDouble();
            if (ev.Payload.TryGetValue("magnitude", out var magVal) && double.TryParse(magVal.ToString(), out double
            {
                if (actualMag < minMag) return false;
            }
            else return false;
        }

        return true;
    }
}

```

3. Database Schema Entity Framework / Migration Script (PostgreSQL / SQL Server)

```

CREATE TABLE alert_rules (
    id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),

```

```
user_id UNIQUEIDENTIFIER NOT NULL,
name NVARCHAR(255) NOT NULL,
category VARCHAR(100) NOT NULL,
conditions_json NVARCHAR(MAX) NOT NULL, -- Condition predicates
is_active BIT DEFAULT 1,
created_at DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET()
);

CREATE TABLE alert_destinations (
id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
alert_rule_id UNIQUEIDENTIFIER FOREIGN KEY REFERENCES alert_rules(id) ON DELETE CASCADE,
channel_type VARCHAR(50) NOT NULL, -- 'email', 'slack'
config_json NVARCHAR(MAX) NOT NULL, -- Webhook URLs, Email addresses encrypted/stored
created_at DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET()
);

CREATE TABLE delivery_logs (
id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
event_id UNIQUEIDENTIFIER NOT NULL,
alert_rule_id UNIQUEIDENTIFIER NOT NULL,
destination_id UNIQUEIDENTIFIER NOT NULL,
status VARCHAR(50) NOT NULL, -- 'PENDING', 'DELIVERED', 'FAILED'
error_message NVARCHAR(MAX) NULL,
retry_count INT DEFAULT 0,
attempted_at DATETIMEOFFSET DEFAULT SYSDATETIMEOFFSET()
);

CREATE INDEX IX_delivery_logs_status_attempted ON delivery_logs (status, attempted_at);
```

4. AI Critical Assessment & Refactoring Log (DECISION_LOG.md)

When executing this plan with AI generation, document findings using this exact format for submission:

Component	AI Code Shortcut / Issue	Technical & Security Risk	Manual Correction Applied
Rule Evaluator	AI generated DynamicLinq or string-evaluated C# code execution for rules.	Remote Code Execution (RCE) / Arbitrary Code Injection risk.	Rewrote rule matching using typed System.Text.Json property parsing and deterministic AST filters.
Slack Channel	AI used synchronous .Result on HTTP POST calls in background thread.	Thread pool starvation and deadlock risk in ASP.NET Core runtime.	Converted to pure async/await pattern passing CancellationToken.
Database Sync	AI put delivery logging inside DB transaction along with external HTTP outbound calls.	Long-held DB connections causing connection pool exhaustion while waiting on Slack/Email APIs.	Separated HTTP execution out of DB transaction context; logged delivery status asynchronously post-delivery.
Admin API	AI omitted [Authorize(Roles = "Admin")] on retry and log query endpoints.	Unauthorized privileges allowing non-admin users to trigger retries and read internal logs.	Enforced JWT RBAC policies and added endpoint authorization filters.

5. Repository File Structure

```
.
├── src/
│   ├── AlertSystem.Core/           # Rules Engine, Driver Interfaces & Models
│   ├── AlertSystem.Infrastructure/  # EF Core DB context, Slack/Email implementations
│   ├── AlertSystem.Worker/         # Hangfire / BackgroundService queue processing
│   └── AlertSystem.Api/            # Admin Controller & Webhook Ingestion API
├── tests/
│   └── AlertSystem.UnitTests/       # Rule Engine & Driver isolate test coverage
├── docs/
│   ├── BRIEF.md                   # Original requirement
│   ├── PLAN.md                    # Plan of attack & design strategy
│   ├── DECISION_LOG.md            # AI Output corrections & security audits
│   └── PROMPT_HISTORY.md          # Chronological prompt trail
├── docker-compose.yml             # Postgres/SQL Server & Redis container setup
└── AlertSystem.sln
```

User prompt: Can I run a C# web app in docker?

Response: Yes, absolutely. Running C# web applications (ASP.NET Core) inside Docker is standard practice for containerized deployments.

The typical workflow uses a **multi-stage Dockerfile**. This compiles and builds the app inside a heavy SDK container, then copies only the built binaries into a lightweight runtime container to keep image sizes small and secure.

Here is a ready-to-use **Dockerfile** for a modern .NET (ASP.NET Core) web application:

```
# 1. Build Stage
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copy project files first and restore dependencies (layer caching optimization)
COPY ["YourApp.csproj", "./"]
RUN dotnet restore "YourApp.csproj"

# Copy remaining source code and publish
COPY . .
RUN dotnet publish "YourApp.csproj" -c Release -o /app/publish /p:UseAppHost=false

# 2. Runtime Stage
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app

# Copy published binaries from the build stage
COPY --from=build /app/publish .

# Expose HTTP port
EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080

ENTRYPOINT ["dotnet", "YourApp.dll"]
```

How to Run It

1. Build the Docker image

```
docker build -t my-dotnet-app .
```

2. Run the container

```
docker run -d -p 8080:8080 --name my-app my-dotnet-app
```

Access the application in your browser at <http://localhost:8080>.

Key Points for C# in Docker

- **Multi-stage builds:** Keeps runtime images clean by omitting build tools, compiler dependencies, and source code.
- **Non-root default user:** Modern .NET runtime images (like .NET 8+) default to non-root execution on port 8080 rather than port 80 for enhanced container security.
- **Docker Compose integration:** If your C# app relies on external services (such as PostgreSQL, SQL Server, or Redis), you can orchestrate them using `docker-compose.yml`:

```
version: '3.8'

services:
  webapi:
    build: .
    ports:
      - "8080:8080"
    environment:
      - ConnectionStrings__DefaultConnection=Server=db;Database=AlertDb;User Id=sa;Password=YourPassword123!;TrustSe
    depends_on:
      - db

  db:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      - ACCEPT_EULA=Y
      - MSSQL_SA_PASSWORD=YourPassword123!
    ports:
      - "1433:1433"
```

User prompt: I started a new Blazor web app. I want to import the news feed from newsapi.org with breaking news titles

Response: 1. Domain Models (Models/NewsModels.cs)

Create response models matching the JSON returned by the NewsAPI `/v2/top-headlines` endpoint:

```
namespace YourAppName.Models;
```

```

public class NewsApiResponse
{
    public string Status { get; set; } = string.Empty;
    public int TotalResults { get; set; }
    public List<NewsArticle> Articles { get; set; } = new();
}

public class NewsArticle
{
    public NewsSource? Source { get; set; }
    public string? Author { get; set; }
    public string Title { get; set; } = string.Empty;
    public string? Description { get; set; }
    public string? Url { get; set; }
    public string? UrlToImage { get; set; }
    public DateTime? PublishedAt { get; set; }
    public string? Content { get; set; }
}

public class NewsSource
{
    public string? Id { get; set; }
    public string? Name { get; set; }
}

```

2. News API Service (Services/NewsApiService.cs)

Create an HTTP client service to handle requests to NewsAPI.

```

using System.Net.Http.Json;
using YourAppName.Models;

namespace YourAppName.Services;

public class NewsApiService
{
    private readonly HttpClient _httpClient;
    private readonly string _apiKey;

    public NewsApiService(HttpClient httpClient, IConfiguration configuration)
    {
        _httpClient = httpClient;
        _apiKey = configuration["NewsApi:ApiKey"]
            ?? throw new ArgumentNullException("NewsApi:ApiKey is not configured.");
    }

    public async Task<List<NewsArticle>> GetTopHeadlinesAsync(string country = "us", string? category = null)
    {
        // Build query string
        var requestUrl = $"v2/top-headlines?country={country}&apiKey={_apiKey}";
        if (!string.IsNullOrEmpty(category))
        {
            requestUrl += "&category={category}";
        }

        try
        {
            var response = await _httpClient.GetFromJsonAsync<NewsApiResponse>(requestUrl);
            return response?.Articles ?? new List<NewsArticle>();
        }
        catch (HttpRequestException ex)
        {
            // Log or handle HTTP errors as needed
            Console.WriteLine($"Error fetching news: {ex.Message}");
            return new List<NewsArticle>();
        }
    }
}

```

3. Register Configuration & Service (Program.cs)

Add API credentials to appsettings.json and register NewsApiService using typed HttpClient.

appsettings.json

```

{
  "Logging": {
    "LogLevel": {

```

```

        "Default": "Information",
        "Microsoft.AspNetCore": "Warning"
    }
},
"AllowedHosts": "*",
"NewsApi": {
    "ApiKey": "YOUR_NEWSAPI_ORG_KEY"
}
}
}

```

Program.cs

```

using YourAppName.Services;

var builder = WebApplication.CreateBuilder(args);

// Register HttpClient with NewsAPI Base URL
builder.Services.AddHttpClient<NewsApiService>(client =>
{
    client.BaseAddress = new Uri("https://newsapi.org/");
    // NewsAPI accepts key either via query string or via User-Agent header
    client.DefaultRequestHeaders.Add("User-Agent", "BlazorNewsApp/1.0");
});

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents(); // or InteractiveWebAssemblyComponents

var app = builder.Build();

// Standard ASP.NET Core Middleware pipeline...
app.UseStaticFiles();
app.UseAntiforgery();

app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

app.Run();

```

4. Blazor View Component (Components/Pages/NewsFeed.razor)

Display headlines in a grid with category filters and loading states:

```

@page "/news"
@using YourAppName.Models
@using YourAppName.Services
@inject NewsApiService NewsService
@attribute [StreamRendering]

<PageTitle>Breaking News Feed</PageTitle>

<h3>Breaking News</h3>

<div class="mb-3">
    <label class="me-2 fw-bold">Select Category:</label>
    <button class="btn btn-outline-primary @(selectedCategory == null ? "active" : "")" @onclick='() => FilterCatego
    <button class="btn btn-outline-primary @(selectedCategory == "business" ? "active" : "")" @onclick='() => Filter
    <button class="btn btn-outline-primary @(selectedCategory == "technology" ? "active" : "")" @onclick='() => Filt
    <button class="btn btn-outline-primary @(selectedCategory == "sports" ? "active" : "")" @onclick='() => FilterCa
</div>

@if (articles == null)
{
    <p><em>Loading headlines...</em></p>
}
else if (!articles.Any())
{
    <p>No headlines found.</p>
}
else
{
    <div class="row row-cols-1 row-cols-md-3 g-4">
        @foreach (var article in articles)
        {
            <div class="col">
                <div class="card h-100 shadow-sm">
                    @if (!string.IsNullOrEmpty(article.UrlToImage))
                    {
                        
}

```

```

        <div class="card-body d-flex flex-column">
            <span class="badge bg-secondary mb-2 text-start w-auto">@article.Source?.Name</span>
            <h5 class="card-title">@article.Title</h5>
            <p class="card-text text-muted small">@article.Description</p>
            <div class="mt-auto">
                <a href="@article.Url" target="_blank" class="btn btn-primary btn-sm">Read Story</a>
                <small class="text-muted d-block mt-2">@article.PublishedAt?.ToString("g")</small>
            </div>
        </div>
    </div>
</div>
}
</div>
}

@code {
    private List<NewsArticle>? articles;
    private string? selectedCategory = null;

    protected override async Task OnInitializedAsync()
    {
        await LoadNewsAsync();
    }

    private async Task FilterCategory(string? category)
    {
        selectedCategory = category;
        articles = null; // trigger loading UI state
        await LoadNewsAsync();
    }

    private async Task LoadNewsAsync()
    {
        articles = await NewsService.GetTopHeadlinesAsync("us", selectedCategory);
    }
}

```

Sources

1. [Top headlines - Documentation - News API](#)

User prompt: In which directory should I create these?

Response: In a standard **Blazor Web App** (ASP.NET Core .NET 8+), organize these files into the following directory structure inside your project root:

```

YourAppProject/
├── Models/
│   └── NewsModels.cs      <-- 1. Domain Models
├── Services/
│   └── NewsApiService.cs  <-- 2. News API Service
├── Components/
│   └── Pages/
│       └── NewsFeed.razor <-- 4. Blazor Component Page
├── Program.cs             <-- 3. Dependency Injection & Pipeline Configuration
└── appsettings.json       <-- 3. API Key Configuration

```

Step-by-Step File Locations

1. Models/NewsModels.cs

- Create a folder named **Models** at the root of your project if it doesn't exist yet.
- Place **NewsApiResponse**, **NewsArticle**, and **NewsSource** inside **Models/NewsModels.cs**.

2. Services/NewsApiService.cs

- Create a folder named **Services** at the root of your project.
- Place your C# HTTP service class inside **Services/NewsApiService.cs**.

3. Program.cs & appsettings.json

- These files already exist at the **root directory** of your Blazor project.
- Add your **"NewsApi": { "ApiKey": "YOUR_KEY" }** section inside **appsettings.json**.
- Register **builder.Services.AddHttpClient<NewsApiService>(...)** directly inside **Program.cs**.

4. Components/Pages/NewsFeed.razor

- In .NET 8+ Blazor Web Apps, pages live in Components/Pages/.
- (If you are using older Blazor Server/WASM, this would be the root Pages/ folder).
- Adding @page "/news" at the top allows users to navigate to http://localhost:port/news.

Note on Namespaces: Ensure the namespace lines in NewsModels.cs and NewsApiService.cs match your project's actual namespace (e.g., namespace MyBlazorApp.Models; and namespace MyBlazorApp.Services;).

User prompt: I can now see the NewsArticles, which is great. I want to store them in the database locally. I only want to store every item once, even if I keep refreshing the page and the NewsArticles are fetched again. Every article should have an id and all records should be soft deletable.

Response: To store the articles locally without duplicates and support soft deletion, you can use **Entity Framework Core (EF Core)** with **SQLite** (or PostgreSQL/SQL Server).

Here is the step-by-step setup to map the model, configure the unique constraint, handle soft deletes, and update your service layer.

Step 1: Install EF Core Packages

Run these commands in your project directory via the terminal or Package Manager Console:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Step 2: Update the NewsArticle Entity (Models/NewsArticle.cs)

Create a dedicated database entity (or update your existing model) with a primary key, a unique index on Url to prevent duplicates, and a IsDeleted flag for soft deletion:

```
namespace YourAppName.Models;

public class NewsArticle
{
    public int Id { get; set; } // Primary Key for local DB

    public string Title { get; set; } = string.Empty;
    public string? Description { get; set; }
    public string? Url { get; set; } // Unique constraint used to prevent duplicate imports
    public string? UrlToImage { get; set; }
    public string? SourceName { get; set; }
    public DateTime? PublishedAt { get; set; }

    // Soft Delete Flag & Timestamp
    public bool IsDeleted { get; set; } = false;
    public DateTime? DeletedAt { get; set; }
    public DateTime ImportedAt { get; set; } = DateTime.UtcNow;
}
```

Step 3: Create the DbContext (Data/AppDbContext.cs)

Create a Data folder in your project root and add AppDbContext.cs:

```
using Microsoft.EntityFrameworkCore;
using YourAppName.Models;

namespace YourAppName.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    public DbSet<NewsArticle> NewsArticles => Set<NewsArticle>();

    protected override OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // 1. Ensure URL is unique so we don't import duplicates
        modelBuilder.Entity<NewsArticle>()
            .HasIndex(a => a.Url)
            .IsUnique();

        // 2. Global Query Filter: Automatically exclude soft-deleted items from normal queries
        modelBuilder.Entity<NewsArticle>()
            .HasQueryFilter(a => !a.IsDeleted);
    }
}
```

```
}
}
```

Step 4: Update NewsApiService to Sync with Database (Services/NewsApiService.cs)

Update NewsApiService to inject IDbContextFactory<AppDbContext>. It fetches headlines from NewsAPI, filters out articles that already exist in the database (or were previously soft-deleted), saves only new ones, and returns the active records from the local database:

```
using System.Net.Http.Json;
using Microsoft.EntityFrameworkCore;
using YourAppName.Data;
using YourAppName.Models;

namespace YourAppName.Services;

public class NewsApiService
{
    private readonly HttpClient _httpClient;
    private readonly string _apiKey;
    private readonly IDbContextFactory<AppDbContext> _dbContextFactory;

    public NewsApiService(
        HttpClient httpClient,
        IConfiguration configuration,
        IDbContextFactory<AppDbContext> dbContextFactory)
    {
        _httpClient = httpClient;
        _apiKey = configuration["NewsApi:ApiKey"]
            ?? throw new ArgumentNullException("NewsApi:ApiKey is not configured.");
        _dbContextFactory = dbContextFactory;
    }

    public async Task SyncAndGetHeadlinesAsync(string country = "us")
    {
        var requestUrl = $"v2/top-headlines?country={country}&apiKey={_apiKey}";

        try
        {
            var response = await _httpClient.GetFromJsonAsync<NewsApiResponseDto>(requestUrl);
            var apiArticles = response?.Articles ?? new List<NewsArticleDto>();

            if (apiArticles.Any())
            {
                await SaveNewArticlesAsync(apiArticles);
            }
        }
        catch (HttpRequestException ex)
        {
            Console.WriteLine($"Error fetching news from API: {ex.Message}");
        }
    }

    private async Task SaveNewArticlesAsync(List<NewsArticleDto> apiArticles)
    {
        using var dbContext = await _dbContextFactory.CreateDbContextAsync();

        // Extract incoming URLs (ignoring articles missing URLs)
        var incomingUrls = apiArticles
            .Where(a => !string.IsNullOrEmpty(a.Url))
            .Select(a => a.Url!)
            .ToList();

        // Query existing URLs (including soft-deleted ones using IgnoreQueryFilters)
        var existingUrls = await dbContext.NewsArticles
            .IgnoreQueryFilters()
            .Where(a => a.Url != null && incomingUrls.Contains(a.Url))
            .Select(a => a.Url!)
            .ToListAsync();

        // Filter out articles already present in the database
        var newArticles = apiArticles
            .Where(a => !string.IsNullOrEmpty(a.Url) && !existingUrls.Contains(a.Url))
            .Select(a => new NewsArticle
            {
                Title = a.Title,
                Description = a.Description,
                Url = a.Url,
                UrlToImage = a.UrlToImage,
                SourceName = a.Source?.Name,
            })
            .ToList();

        dbContext.NewsArticles.AddRange(newArticles);
        await dbContext.SaveChangesAsync();
    }
}
```

```

        PublishedAt = a.PublishedAt,
        ImportedAt = DateTime.UtcNow,
        IsDeleted = false
    })
    .ToList();

    if (newArticles.Any())
    {
        dbContext.NewsArticles.AddRange(newArticles);
        await dbContext.SaveChangesAsync();
    }
}

// Get all active (non-deleted) articles from local DB
public async Task<List<NewsArticle>> GetLocalArticlesAsync()
{
    using var dbContext = await _dbContextFactory.CreateDbContextAsync();
    return await dbContext.NewsArticles
        .OrderByDescending(a => a.PublishedAt ?? a.ImportedAt)
        .ToListAsync();
}

// Soft delete an article by ID
public async Task SoftDeleteArticleAsync(int id)
{
    using var dbContext = await _dbContextFactory.CreateDbContextAsync();
    var article = await dbContext.NewsArticles.FindAsync(id);
    if (article != null)
    {
        article.IsDeleted = true;
        article.DeletedAt = DateTime.UtcNow;
        await dbContext.SaveChangesAsync();
    }
}
}

// API Response DTOs
public class NewsApiResponseDto
{
    public List<NewsArticleDto> Articles { get; set; } = new();
}

public class NewsArticleDto
{
    public NewsSourceDto? Source { get; set; }
    public string Title { get; set; } = string.Empty;
    public string? Description { get; set; }
    public string? Url { get; set; }
    public string? UrlToImage { get; set; }
    public DateTime? PublishedAt { get; set; }
}

public class NewsSourceDto
{
    public string? Name { get; set; }
}

```

Step 5: Register DbContext & Apply Migrations (Program.cs)

Add the SQLite DbContextFactory to Program.cs and ensure the database is created on app startup:

```

using Microsoft.EntityFrameworkCore;
using YourAppName.Data;
using YourAppName.Services;

var builder = WebApplication.CreateBuilder(args);

// Register SQLite DbContextFactory for Blazor Server concurrency safety
builder.Services.AddDbContextFactory<AppDbContext>(options =>
    options.UseSqlite("Data Source=news.db"));

builder.Services.AddHttpClient<NewsApiService>(client =>
{
    client.BaseAddress = new Uri("https://newsapi.org/");
    client.DefaultRequestHeaders.Add("User-Agent", "BlazorNewsApp/1.0");
});

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

```

```

var app = builder.Build();

// Auto-create SQLite DB schema on startup if it doesn't exist
using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    dbContext.Database.EnsureCreated();
}

app.UseStaticFiles();
app.UseAntiforgery();

app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

app.Run();

```

Step 6: Update the Blazor Component (Components/Pages/NewsFeed.razor)

Update the UI to fetch local articles and add a **Soft Delete** button to each card:

```

@page "/news"
@using YourAppName.Models
@using YourAppName.Services
@inject NewsApiService NewsService
@attribute [StreamRendering]

<PageTitle>Local Breaking News</PageTitle>

<div class="d-flex justify-content-between align-items-center mb-3">
    <h3>Local Breaking News Feed</h3>
    <button class="btn btn-outline-secondary" @onclick="SyncNewsAsync" disabled="@isSyncing">
        @(isSyncing ? "Syncing..." : "Sync Latest Headlines")
    </button>
</div>

@if (articles == null)
{
    <p><em>Loading local articles...</em></p>
}
else if (!articles.Any())
{
    <p>No articles stored. Click "Sync Latest Headlines" to pull news from NewsAPI.</p>
}
else
{
    <div class="row row-cols-1 row-cols-md-3 g-4">
        @foreach (var article in articles)
        {
            <div class="col">
                <div class="card h-100 shadow-sm">
                    @if (!string.IsNullOrEmpty(article.UrlToImage))
                    {
                        
                        <span class="badge bg-secondary mb-2 text-start w-auto">@article.SourceName</span>
                        <h5 class="card-title">@article.Title</h5>
                        <p class="card-text text-muted small">@article.Description</p>
                        <div class="mt-auto d-flex justify-content-between align-items-center">
                            <a href="@article.Url" target="_blank" class="btn btn-primary btn-sm">Read</a>
                            <button class="btn btn-outline-danger btn-sm" @onclick="() => SoftDeleteAsync(article.Id
                                Delete
                            </button>
                        </div>
                    </div>
                </div>
            </div>
        }
    </div>
}

@code {
    private List<NewsArticle>? articles;
    private bool isSyncing = false;

    protected override async Task OnInitializedAsync()
    {

```

```

        await RefreshLocalArticlesAsync();
    }

    private async Task SyncNewsAsync()
    {
        isSyncing = true;
        await NewsService.SyncAndGetHeadlinesAsync();
        await RefreshLocalArticlesAsync();
        isSyncing = false;
    }

    private async Task SoftDeleteAsync(int id)
    {
        await NewsService.SoftDeleteArticleAsync(id);
        await RefreshLocalArticlesAsync();
    }

    private async Task RefreshLocalArticlesAsync()
    {
        articles = await NewsService.GetLocalArticlesAsync();
    }
}

```

User prompt: Override is not valid for this item, and it must have a return type: protected override OnModelCreating(ModelBuilder modelBuilder) { base.OnModelCreating(modelBuilder); // 1. Ensure URL is unique so we don't import duplicates modelBuilder.Entity<NewsArticle>().HasIndex(a => a.Url).IsUnique(); // 2. Global Query Filter: Automatically exclude soft-deleted items from normal queries modelBuilder.Entity<NewsArticle>().HasQueryFilter(a => !a.IsDeleted); }

Response: In C#, methods overridden from a base class must explicitly specify their return type. OnModelCreating returns void.

The Fix

Add void before OnModelCreating:

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    // 1. Ensure URL is unique so we don't import duplicates
    modelBuilder.Entity<NewsArticle>()
        .HasIndex(a => a.Url)
        .IsUnique();

    // 2. Global Query Filter: Automatically exclude soft-deleted items from normal queries
    modelBuilder.Entity<NewsArticle>()
        .HasQueryFilter(a => !a.IsDeleted);
}

```

Why this happened

In C#, every method declaration requires a return type (or void if it returns nothing). The override keyword modifies the method signature, but it cannot replace the return type specification.

User prompt: The sync latest headlines does not work in the browser. No request is sent. Do I need to create db tables? Also, my app's name is Sonrisa, please use that.

Response: Yes, EF Core with SQLite creates the database file (news.db), but depending on how SQLite handles initialization, missing tables or unhandled UI events can cause silent failures.

Here is why no request is triggering when you click the button in **Sonrisa** and how to fix both issues.

Reason 1: Blazor Interactive Render Mode is Disabled on the Page

In .NET 8 Blazor Web Apps, pages default to **Static Server Rendering (SSR)** unless explicitly told to be interactive. In SSR mode, button @onclick handlers do **not** trigger C# code or HTTP calls in the browser because no WebSockets or WebAssembly connection is active.

At the top of Components/Pages/NewsFeed.razor, update your render mode directive to enable interactivity:

```

@page "/news"
@using Sonrisa.Models
@using Sonrisa.Services
@inject NewsApiService NewsService
@rendermode InteractiveServer

<PageTitle>Sonrisa - Local Breaking News</PageTitle>

```

Note: Ensure `@rendermode InteractiveServer` is present at the top of `NewsFeed.razor`.

Reason 2: Creating Database Tables Automatically

If you used `dbContext.Database.EnsureCreated()` in `Program.cs`, EF Core creates the `news.db` file and all tables (`NewsArticles`) automatically on application startup. However, if the `news.db` file was created prior to defining `DbSet<NewsArticle>`, EF Core sees the file exists and skips creating the missing tables.

Update your `Program.cs` startup logic in **Sonrisa** to use `Database.Migrate()` or delete the empty `news.db` file so it recreates properly with all required tables:

Updated Program.cs for Sonrisa

```
using Microsoft.EntityFrameworkCore;
using Sonrisa.Data;
using Sonrisa.Services;

var builder = WebApplication.CreateBuilder(args);

// Register SQLite DbContextFactory for Sonrisa
builder.Services.AddDbContextFactory<AppDbContext>(options =>
    options.UseSqlite("Data Source=news.db"));

builder.Services.AddHttpClient<NewsApiService>(client =>
{
    client.BaseAddress = new Uri("https://newsapi.org/");
    client.DefaultRequestHeaders.Add("User-Agent", "SonrisaNewsApp/1.0");
});

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

var app = builder.Build();

// Ensure DB schema is created on startup
using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    // EnsureCreated creates the database file and all tables defined in AppDbContext
    dbContext.Database.EnsureCreated();
}

app.UseStaticFiles();
app.UseAntiforgery();

app.MapRazorComponents<Sonrisa.Components.App>()
    .AddInteractiveServerRenderMode();

app.Run();
```

Troubleshooting Checklist for Sonrisa

1. **Delete existing news.db file** in your project root and restart the application so `EnsureCreated()` builds fresh tables.
2. Ensure **App.razor** or **Program.cs** has interactivity registered via `builder.Services.AddRazorComponents().AddInteractiveServerComponents()` and `app.MapRazorComponents<App>().AddInteractiveServerRenderMode()`.
3. Check the Browser Developer Tools Console (F12 -> Console tab) to ensure the Blazor WebSocket (`_framework/blazor.web.js`) connects without errors.

User prompt: I want to add a new page where users can subscribe or unsubscribe from notifications of these news articles. The form should have 3 input fields: a field for an email address, a checkbox for an email notification and a checkbox for a Slack notification. It should create and save the user in the database in a new field. If both checkboxes are unset, we should remove the user from the database. If the checkboxes are set, I will want to send the top messages via email and/or Slack to the subscribed users. Please prepare the Slack integration, but we'll implement the details in the next step.

Response: To add notification subscriptions to **Sonrisa**, we will create a `Subscriber` model, configure EF Core, build a `SubscriberService`, and create a Blazor subscription page (`/subscribe`).

Step 1: Create the Subscriber Model (`Models/Subscriber.cs`)

Create a `Subscriber` entity with soft deletion support and flags for Email and Slack notifications:

```
namespace Sonrisa.Models;

public class Subscriber
```

```
{
    public int Id { get; set; }
    public string Email { get; set; } = string.Empty;
    public bool NotifyViaEmail { get; set; }
    public bool NotifyViaSlack { get; set; }
    public string? SlackWebhookUrl { get; set; } // Prepared for Slack integration in the next step

    public bool IsDeleted { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime? UpdatedAt { get; set; }
}
```

Step 2: Update ApplicationDbContext (Data/AppDbContext.cs)

Add the Subscribers DbSet, configure a unique index on Email, and add a global query filter for soft delete:

```
using Microsoft.EntityFrameworkCore;
using Sonrisa.Models;

namespace Sonrisa.Data;

public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
    {
    }

    public DbSet<NewsArticle> NewsArticles => Set<NewsArticle>();
    public DbSet<Subscriber> Subscribers => Set<Subscriber>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // NewsArticle Configuration
        modelBuilder.Entity<NewsArticle>()
            .HasIndex(a => a.Url)
            .IsUnique();

        modelBuilder.Entity<NewsArticle>()
            .HasQueryFilter(a => !a.IsDeleted);

        // Subscriber Configuration
        modelBuilder.Entity<Subscriber>()
            .HasIndex(s => s.Email)
            .IsUnique();

        modelBuilder.Entity<Subscriber>()
            .HasQueryFilter(s => !s.IsDeleted);
    }
}
```

Step 3: Create the Subscriber Service (Services/SubscriberService.cs)

This service handles saving, updating, and removing subscribers when both channels are unchecked:

```
using Microsoft.EntityFrameworkCore;
using Sonrisa.Data;
using Sonrisa.Models;

namespace Sonrisa.Services;

public class SubscriberService
{
    private readonly IDbContextFactory<ApplicationDbContext> _dbContextFactory;

    public SubscriberService(IDbContextFactory<ApplicationDbContext> dbContextFactory)
    {
        _dbContextFactory = dbContextFactory;
    }

    public async Task<Subscriber?> GetByEmailAsync(string email)
    {
        using var db = await _dbContextFactory.CreateDbContextAsync();
        return await db.Subscribers.FirstOrDefaultAsync(s => s.Email.ToLower() == email.ToLower());
    }

    public async Task SaveSubscriptionAsync(string email, bool notifyViaEmail, bool notifyViaSlack, string? slackWeb
```

```

{
    using var db = await _dbContextFactory.CreateDbContextAsync();
    var normalizedEmail = email.Trim().ToLower();

    // Check for existing record (including soft-deleted ones)
    var existing = await db.Subscribers
        .IgnoreQueryFilters()
        .FirstOrDefaultAsync(s => s.Email.ToLower() == normalizedEmail);

    // Rule: If both checkboxes are unchecked, remove (or soft-delete) the subscriber
    if (!notifyViaEmail && !notifyViaSlack)
    {
        if (existing != null && !existing.IsDeleted)
        {
            existing.IsDeleted = true;
            existing.UpdatedAt = DateTime.UtcNow;
            await db.SaveChangesAsync();
        }
        return;
    }

    if (existing == null)
    {
        // Create new subscriber
        var subscriber = new Subscriber
        {
            Email = normalizedEmail,
            NotifyViaEmail = notifyViaEmail,
            NotifyViaSlack = notifyViaSlack,
            SlackWebhookUrl = slackWebhookUrl,
            CreatedAt = DateTime.UtcNow
        };
        db.Subscribers.Add(subscriber);
    }
    else
    {
        // Restore if previously deleted and update preferences
        existing.IsDeleted = false;
        existing.NotifyViaEmail = notifyViaEmail;
        existing.NotifyViaSlack = notifyViaSlack;
        if (!string.IsNullOrEmpty(slackWebhookUrl))
        {
            existing.SlackWebhookUrl = slackWebhookUrl;
        }
        existing.UpdatedAt = DateTime.UtcNow;
    }

    await db.SaveChangesAsync();
}

public async Task<List<Subscriber>> GetActiveSubscribersAsync()
{
    using var db = await _dbContextFactory.CreateDbContextAsync();
    return await db.Subscribers.ToListAsync();
}
}

```

Step 4: Register SubscriberService in Program.cs

Add SubscriberService to your dependency injection container:

```

// Program.cs
builder.Services.AddTransient<SubscriberService>();

```

(Note: Since you modified AppDbContext, delete your existing news.db file in the project root so EnsureCreated() recreates the database with the new Subscribers table).

Step 5: Build Subscription Page (Components/Pages/Subscribe.razor)

Create the form component allowing users to set or clear notification preferences:

```

@page "/subscribe"
@using Sonrisa.Services
@inject SubscriberService SubscriberService
@rendermode InteractiveServer

<PageTitle>Sonrisa - Notification Preferences</PageTitle>

<div class="row justify-content-center">

```

```

<div class="col-md-6">
  <div class="card shadow-sm mt-4">
    <div class="card-header bg-primary text-white">
      <h4 class="mb-0">News Notification Alert Setup</h4>
    </div>
    <div class="card-body">
      @if (!string.IsNullOrEmpty(statusMessage))
      {
        <div class="alert @statusCssClass alert-dismissible fade show" role="alert">
          @statusMessage
        </div>
      }

      <EditForm Model="model" OnValidSubmit="HandleSubmitAsync">
        <DataAnnotationsValidator />

        <div class="mb-3">
          <label for="email" class="form-label">Email Address</label>
          <InputText id="email" class="form-control" @bind-Value="model.Email" placeholder="user@examp
          <ValidationMessage For="() => model.Email" />
        </div>

        <div class="form-check mb-2">
          <InputCheckbox id="notifyEmail" class="form-check-input" @bind-Value="model.NotifyViaEmail"
          <label class="form-check-label" for="notifyEmail">
            Receive Breaking News via Email
          </label>
        </div>

        <div class="form-check mb-3">
          <InputCheckbox id="notifySlack" class="form-check-input" @bind-Value="model.NotifyViaSlack"
          <label class="form-check-label" for="notifySlack">
            Receive Breaking News via Slack
          </label>
        </div>

        @if (model.NotifyViaSlack)
        {
          <div class="mb-3 border p-3 rounded bg-light">
            <label for="slackWebhook" class="form-label">Slack Incoming Webhook URL</label>
            <InputText id="slackWebhook" class="form-control" @bind-Value="model.SlackWebhookUrl" pl
            <small class="text-muted">Prepared for Slack dispatch integration.</small>
          </div>
        }

        <div class="d-grid gap-2">
          <button type="submit" class="btn btn-primary" disabled="@isProcessing">
            @(isProcessing ? "Saving..." : "Save Preferences")
          </button>
        </div>
      </EditForm>
    </div>
  </div>
</div>
</div>

@code {
  private SubscriptionModel model = new();
  private string? statusMessage;
  private string statusCssClass = "alert-info";
  private bool isProcessing = false;

  private async Task LookupExistingSubscriber()
  {
    if (string.IsNullOrEmpty(model.Email)) return;

    var existing = await SubscriberService.GetByEmailAsync(model.Email);
    if (existing != null)
    {
      model.NotifyViaEmail = existing.NotifyViaEmail;
      model.NotifyViaSlack = existing.NotifyViaSlack;
      model.SlackWebhookUrl = existing.SlackWebhookUrl;
      statusMessage = "Found existing subscription. Preferences loaded.";
      statusCssClass = "alert-info";
    }
  }

  private async Task HandleSubmitAsync()
  {

```

```

        isProcessing = true;
        statusMessage = null;

        await SubscriberService.SaveSubscriptionAsync(
            model.Email,
            model.NotifyViaEmail,
            model.NotifyViaSlack,
            model.SlackWebhookUrl
        );

        if (!model.NotifyViaEmail && !model.NotifyViaSlack)
        {
            statusMessage = "Unsubscribed successfully. Your record has been removed.";
            statusCssClass = "alert-warning";
        }
        else
        {
            statusMessage = "Preferences saved successfully!";
            statusCssClass = "alert-success";
        }

        isProcessing = false;
    }

    public class SubscriptionModel
    {
        [System.ComponentModel.DataAnnotations.Required]
        [System.ComponentModel.DataAnnotations.EmailAddress]
        public string Email { get; set; } = string.Empty;

        public bool NotifyViaEmail { get; set; }
        public bool NotifyViaSlack { get; set; }
        public string? SlackWebhookUrl { get; set; }
    }
}

```

User prompt: First: please create the table for Subscribers. Let's continue with Slack.

Response: Here is the SQL table script for Subscribers, followed by the complete Slack notification engine implementation for **Sonrisa**.

1. SQL Script for Subscribers Table

If you need to manually execute the SQL create statement in SQLite (or via database tools), run the following:

```

CREATE TABLE IF NOT EXISTS Subscribers (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    Email TEXT NOT NULL,
    NotifyViaEmail INTEGER NOT NULL DEFAULT 0,
    NotifyViaSlack INTEGER NOT NULL DEFAULT 0,
    SlackWebhookUrl TEXT NULL,
    IsDeleted INTEGER NOT NULL DEFAULT 0,
    CreatedAt TEXT NOT NULL,
    UpdatedAt TEXT NULL
);

-- Unique constraint on Email to prevent duplicate rows
CREATE UNIQUE INDEX IF NOT EXISTS IX_Subscribers_Email ON Subscribers(Email);

```

(Note: If you run `dbContext.Database.EnsureCreated()` after deleting `news.db`, EF Core will automatically generate this table for you).

2. Slack Notification Engine Architecture

We will implement the **Strategy/Driver Pattern** for notifications. This creates a clean `ISlackNotificationService` that builds formatted Slack Block Kit messages and posts them via `HttpClient`.

Step 1: Create the Slack Notification Service (`Services/SlackNotificationService.cs`)

Create `Services/SlackNotificationService.cs` to format and deliver news alerts to Slack Webhooks:

```

using System.Text;
using System.Text.Json;
using Sonrisa.Models;

namespace Sonrisa.Services;

public class SlackNotificationService
{

```

```

private readonly HttpClient _httpClient;

public SlackNotificationService(HttpClient httpClient)
{
    _httpClient = httpClient;
}

public async Task<bool> SendNewsNotificationAsync(string webhookUrl, List<NewsArticle> articles)
{
    if (string.IsNullOrEmpty(webhookUrl) || !articles.Any())
        return false;

    // Build Slack Payload using Slack Block Kit JSON format
    var blocks = new List<object>
    {
        new
        {
            type = "header",
            text = new
            {
                type = "plain_text",
                text = "🔔 Sonrisa Breaking News Alert",
                emoji = true
            }
        },
        new { type = "divider" }
    };

    foreach (var article in articles.Take(5)) // Top 5 headlines
    {
        var articleText = $"*{<{article.Url}>{<{article.Title}>*\n" +
            $"_Source: {article.SourceName ?? "Unknown"}_ | Published: {article.PublishedAt?.ToString()}\n" +
            $"Description: {article.Description}";

        blocks.Add(new
        {
            type = "section",
            text = new
            {
                type = "mrkdown",
                text = articleText
            }
        });

        blocks.Add(new { type = "divider" });
    }

    var payload = new { blocks };
    var json = JsonSerializer.Serialize(payload);
    var content = new StringContent(json, Encoding.UTF8, "application/json");

    try
    {
        var response = await _httpClient.PostAsync(webhookUrl, content);
        return response.IsSuccessStatusCode;
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[Slack Dispatch Error] {ex.Message}");
        return false;
    }
}
}

```

Step 2: Create Notification Dispatcher (Services/NotificationDispatcherService.cs)

This service orchestrates sending top headlines to all active subscribers who opted into Slack notifications:

```

using Microsoft.EntityFrameworkCore;
using Sonrisa.Data;
using Sonrisa.Models;

namespace Sonrisa.Services;

public class NotificationDispatcherService
{
    private readonly IDbContextFactory<AppDbContext> _dbContextFactory;
    private readonly SlackNotificationService _slackService;
}

```

```

public NotificationDispatcherService(
    IDbContextFactory<AppDbContext> dbContextFactory,
    SlackNotificationService slackService)
{
    _dbContextFactory = dbContextFactory;
    _slackService = slackService;
}

public async Task<int> DispatchTopHeadlinesToSlackAsync(int topCount = 3)
{
    using var db = await _dbContextFactory.CreateDbContextAsync();

    // 1. Fetch latest active news articles
    var recentArticles = await db.NewsArticles
        .OrderByDescending(a => a.PublishedAt ?? a.ImportedAt)
        .Take(topCount)
        .ToListAsync();

    if (!recentArticles.Any())
        return 0;

    // 2. Fetch subscribers opted into Slack with valid webhooks
    var slackSubscribers = await db.Subscribers
        .Where(s => s.NotifyViaSlack && !string.IsNullOrEmpty(s.SlackWebhookUrl))
        .ToListAsync();

    int successCount = 0;

    // 3. Dispatch to subscribers
    foreach (var subscriber in slackSubscribers)
    {
        bool sent = await _slackService.SendNewsNotificationAsync(subscriber.SlackWebhookUrl!, recentArticles);
        if (sent)
        {
            successCount++;
        }
    }

    return successCount;
}
}

```

Step 3: Register Services in Program.cs

Add the Slack HTTP client and Dispatcher service to Program.cs:

```

using Sonrisa.Services;

// Register Slack Service with HttpClient
builder.Services.AddHttpClient<SlackNotificationService>();

// Register Dispatcher
builder.Services.AddTransient<NotificationDispatcherService>();

```

Step 4: Add Manual Trigger Button in News Feed (Components/Pages/NewsFeed.razor)

Add a **"Send Top News to Slack"** action button on the /news page so you can test and trigger notifications instantly:

```

@inject NotificationDispatcherService DispatcherService

<div class="d-flex gap-2">
    <button class="btn btn-outline-secondary" @onclick="SyncNewsAsync" disabled="@isSyncing">
        @(isSyncing ? "Syncing..." : "Sync Latest Headlines")
    </button>
    <button class="btn btn-outline-primary" @onclick="DispatchSlackAsync" disabled="@isDispatching">
        @(isDispatching ? "Sending..." : "Dispatch to Slack Subscribers")
    </button>
</div>

@if (!string.IsNullOrEmpty(dispatchStatus))
{
    <div class="alert alert-info mt-2 mb-0 py-2">
        @dispatchStatus
    </div>
}

@code {
    private bool isDispatching = false;
    private string? dispatchStatus;
}

```

```
private async Task DispatchSlackAsync()
{
    isDispatching = true;
    dispatchStatus = null;

    int notifiedCount = await DispatcherService.DispatchTopHeadlinesToSlackAsync(topCount: 3);
    dispatchStatus = $"Successfully dispatched breaking news to {notifiedCount} Slack subscriber(s)!";

    isDispatching = false;
}
}
```

User prompt: How do I build an Admin view in Sonrisa to inspect active subscribers and notification log status?

Response: To give **Sonrisa** full administrative oversight, we will build a dedicated Admin View (/admin) to inspect active subscribers, view soft-deleted records, track delivery logs, and manually trigger notification dispatches.

1. SQL Schema Update for Delivery Logs

First, let's create a table to track notification dispatch attempts and results.

```
CREATE TABLE IF NOT EXISTS DeliveryLogs (
    Id INTEGER PRIMARY KEY AUTOINCREMENT,
    ChannelType TEXT NOT NULL, -- 'Slack' or 'Email'
    Recipient TEXT NOT NULL,    -- Email address or Webhook URL
    ArticleCount INTEGER NOT NULL,
    Status TEXT NOT NULL,      -- 'SUCCESS' or 'FAILED'
    ErrorMessage TEXT NULL,
    AttemptedAt TEXT NOT NULL
);

CREATE INDEX IF NOT EXISTS IX_DeliveryLogs_Status_AttemptedAt ON DeliveryLogs(Status, AttemptedAt);
```

2. Delivery Log Model (Models/DeliveryLog.cs)

Create Models/DeliveryLog.cs:

```
namespace Sonrisa.Models;

public class DeliveryLog
{
    public int Id { get; set; }
    public string ChannelType { get; set; } = string.Empty; // "Slack" or "Email"
    public string Recipient { get; set; } = string.Empty;
    public int ArticleCount { get; set; }
    public string Status { get; set; } = string.Empty; // "SUCCESS" or "FAILED"
    public string? ErrorMessage { get; set; }
    public DateTime AttemptedAt { get; set; } = DateTime.UtcNow;
}
```

3. Update AppDbContext.cs (Data/AppDbContext.cs)

Add DeliveryLogs to AppDbContext:

```
using Microsoft.EntityFrameworkCore;
using Sonrisa.Models;

namespace Sonrisa.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    public DbSet<NewsArticle> NewsArticles => Set<NewsArticle>();
    public DbSet<Subscriber> Subscribers => Set<Subscriber>();
    public DbSet<DeliveryLog> DeliveryLogs => Set<DeliveryLog>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // NewsArticle Configuration
        modelBuilder.Entity<NewsArticle>()
```

```

        .HasIndex(a => a.Url)
        .IsUnique();
modelBuilder.Entity<NewsArticle>()
    .HasQueryFilter(a => !a.IsDeleted);

// Subscriber Configuration
modelBuilder.Entity<Subscriber>()
    .HasIndex(s => s.Email)
    .IsUnique();
modelBuilder.Entity<Subscriber>()
    .HasQueryFilter(s => !s.IsDeleted);

// DeliveryLog Indexing
modelBuilder.Entity<DeliveryLog>()
    .HasIndex(l => new { l.Status, l.AttemptedAt });
    }
}

```

4. Log Execution in NotificationDispatcherService.cs

Update NotificationDispatcherService.cs to write delivery logs whenever notifications are dispatched:

```

using Microsoft.EntityFrameworkCore;
using Sonrisa.Data;
using Sonrisa.Models;

namespace Sonrisa.Services;

public class NotificationDispatcherService
{
    private readonly IDbContextFactory<AppDbContext> _dbContextFactory;
    private readonly SlackNotificationService _slackService;

    public NotificationDispatcherService(
        IDbContextFactory<AppDbContext> dbContextFactory,
        SlackNotificationService slackService)
    {
        _dbContextFactory = dbContextFactory;
        _slackService = slackService;
    }

    public async Task<int> DispatchTopHeadlinesToSlackAsync(int topCount = 3)
    {
        using var db = await _dbContextFactory.CreateDbContextAsync();

        var recentArticles = await db.NewsArticles
            .OrderByDescending(a => a.PublishedAt ?? a.ImportedAt)
            .Take(topCount)
            .ToListAsync();

        if (!recentArticles.Any())
            return 0;

        var slackSubscribers = await db.Subscribers
            .Where(s => s.NotifyViaSlack && !string.IsNullOrEmpty(s.SlackWebhookUrl))
            .ToListAsync();

        int successCount = 0;

        foreach (var subscriber in slackSubscribers)
        {
            bool sent = await _slackService.SendNewsNotificationAsync(subscriber.SlackWebhookUrl!, recentArticles);

            var log = new DeliveryLog
            {
                ChannelType = "Slack",
                Recipient = subscriber.Email,
                ArticleCount = recentArticles.Count,
                Status = sent ? "SUCCESS" : "FAILED",
                ErrorMessage = sent ? null : "Failed to deliver payload to Slack webhook.",
                AttemptedAt = DateTime.UtcNow
            };

            db.DeliveryLogs.Add(log);

            if (sent)
            {
                successCount++;
            }
        }
    }
}

```

```

    }

    await db.SaveChangesAsync();
    return successCount;
}
}

```

5. Admin Service (Services/AdminService.cs)

Create a service to manage admin operations like metrics calculation, fetching logs, and soft-delete/restore operations:

```

using Microsoft.EntityFrameworkCore;
using Sonrisa.Data;
using Sonrisa.Models;

namespace Sonrisa.Services;

public class AdminService
{
    private readonly IDbContextFactory<AppDbContext> _dbContextFactory;

    public AdminService(IDbContextFactory<AppDbContext> dbContextFactory)
    {
        _dbContextFactory = dbContextFactory;
    }

    public async Task<AdminMetrics> GetMetricsAsync()
    {
        using var db = await _dbContextFactory.CreateDbContextAsync();

        return new AdminMetrics
        {
            TotalActiveSubscribers = await db.Subscribers.CountAsync(),
            SlackSubscribersCount = await db.Subscribers.CountAsync(s => s.NotifyViaSlack),
            EmailSubscribersCount = await db.Subscribers.CountAsync(s => s.NotifyViaEmail),
            TotalDispatches = await db.DeliveryLogs.CountAsync(),
            SuccessfulDispatches = await db.DeliveryLogs.CountAsync(l => l.Status == "SUCCESS"),
            FailedDispatches = await db.DeliveryLogs.CountAsync(l => l.Status == "FAILED")
        };
    }

    public async Task<List<Subscriber>> GetAllSubscribersAsync(bool includeDeleted = false)
    {
        using var db = await _dbContextFactory.CreateDbContextAsync();
        var query = db.Subscribers.AsQueryable();

        if (includeDeleted)
        {
            query = query.IgnoreQueryFilters();
        }

        return await query.OrderByDescending(s => s.CreatedAt).ToListAsync();
    }

    public async Task<List<DeliveryLog>> GetRecentDeliveryLogsAsync(int take = 50)
    {
        using var db = await _dbContextFactory.CreateDbContextAsync();
        return await db.DeliveryLogs
            .OrderByDescending(l => l.AttemptedAt)
            .Take(take)
            .ToListAsync();
    }

    public async Task ToggleSubscriberSoftDeleteAsync(int id)
    {
        using var db = await _dbContextFactory.CreateDbContextAsync();
        var sub = await db.Subscribers.IgnoreQueryFilters().FirstOrDefaultAsync(s => s.Id == id);

        if (sub != null)
        {
            sub.IsDeleted = !sub.IsDeleted;
            sub.UpdatedAt = DateTime.UtcNow;
            await db.SaveChangesAsync();
        }
    }
}

public class AdminMetrics
{

```

```

    public int TotalActiveSubscribers { get; set; }
    public int SlackSubscribersCount { get; set; }
    public int EmailSubscribersCount { get; set; }
    public int TotalDispatches { get; set; }
    public int SuccessfulDispatches { get; set; }
    public int FailedDispatches { get; set; }
}

```

Register this service in Program.cs:

```
builder.Services.AddTransient<AdminService>();
```

6. Admin View Component (Components/Pages/Admin.razor)

Create Components/Pages/Admin.razor:

```

@page "/admin"
@using Sonrisa.Models
@using Sonrisa.Services
@inject AdminService AdminService
@inject NotificationDispatcherService DispatcherService
@rendermode InteractiveServer

<PageTitle>Sonrisa - Admin Dashboard</PageTitle>

<div class="container my-4">
    <div class="d-flex justify-content-between align-items-center mb-4">
        <h2>Admin Dashboard</h2>
        <button class="btn btn-primary" @onclick="DispatchSlackAsync" disabled="@isDispatching">
            @(isDispatching ? "Dispatching..." : "Trigger Slack Dispatch Now")
        </button>
    </div>

    @if (!string.IsNullOrEmpty(actionStatus))
    {
        <div class="alert alert-info alert-dismissible fade show" role="alert">
            @actionStatus
        </div>
    }

    @if (metrics != null)
    {
        <div class="row g-3 mb-4">
            <div class="col-md-3">
                <div class="card bg-light shadow-sm text-center p-3">
                    <span class="text-muted small">Active Subscribers</span>
                    <h3 class="mb-0 text-primary">@metrics.TotalActiveSubscribers</h3>
                </div>
            </div>
            <div class="col-md-3">
                <div class="card bg-light shadow-sm text-center p-3">
                    <span class="text-muted small">Slack / Email Opt-ins</span>
                    <h3 class="mb-0 text-dark">@metrics.SlackSubscribersCount / @metrics.EmailSubscribersCount</h3>
                </div>
            </div>
            <div class="col-md-3">
                <div class="card bg-light shadow-sm text-center p-3">
                    <span class="text-muted small">Successful Dispatches</span>
                    <h3 class="mb-0 text-success">@metrics.SuccessfulDispatches</h3>
                </div>
            </div>
            <div class="col-md-3">
                <div class="card bg-light shadow-sm text-center p-3">
                    <span class="text-muted small">Failed Dispatches</span>
                    <h3 class="mb-0 text-danger">@metrics.FailedDispatches</h3>
                </div>
            </div>
        </div>
    }

    <!-- Tabbed Navigation -->
    <ul class="nav nav-tabs mb-3" id="adminTab">
        <li class="nav-item">
            <button class="nav-link @(activeTab == "subscribers" ? "active" : "")" @onclick='() => SetTab("subscribe Subscribers Management")'>
                Subscribers Management
            </button>
        </li>
        <li class="nav-item">
            <button class="nav-link @(activeTab == "logs" ? "active" : "")" @onclick='() => SetTab("logs")'>
                Logs
            </button>
        </li>
    </ul>

```

```

        Delivery Logs
      </button>
    </li>
  </ul>

  <!-- Tab Content -->
  @if (activeTab == "subscribers")
  {
    <div class="card shadow-sm p-3">
      <div class="d-flex justify-content-between align-items-center mb-3">
        <h5 class="mb-0">Subscribers</h5>
        <div class="form-check form-switch">
          <input class="form-check-input" type="checkbox" id="showDeletedSwitch" @bind="showDeleted" @bind
            <label class="form-check-label" for="showDeletedSwitch">Include Unsubscribed/Deleted</label>
        </div>
      </div>

      @if (subscribers == null)
      {
        <p><em>Loading subscribers...</em></p>
      }
      else
      {
        <div class="table-responsive">
          <table class="table table-hover align-middle">
            <thead>
              <tr>
                <th>Email</th>
                <th>Channels</th>
                <th>Created At</th>
                <th>Status</th>
                <th>Action</th>
              </tr>
            </thead>
            <tbody>
              @foreach (var sub in subscribers)
              {
                <tr class="@((sub.IsDeleted ? "table-secondary text-muted" : ""))">
                  <td>@sub.Email</td>
                  <td>
                    @if (sub.NotifyViaEmail) { <span class="badge bg-info text-dark me-1">Email<
                    @if (sub.NotifyViaSlack) { <span class="badge bg-primary">Slack</span> }
                  </td>
                  <td>@sub.CreatedAt.ToString("g")</td>
                  <td>
                    @if (sub.IsDeleted)
                    {
                      <span class="badge bg-danger">Deleted</span>
                    }
                    else
                    {
                      <span class="badge bg-success">Active</span>
                    }
                  </td>
                  <td>
                    <button class="btn btn-sm @(sub.IsDeleted ? "btn-outline-success" : "btn-out
                      @onclick="() => ToggleSoftDeleteAsync(sub.Id)">
                      @(sub.IsDeleted ? "Restore" : "Soft Delete")
                    </button>
                  </td>
                </tr>
              }
            </tbody>
          </table>
        </div>
      }
    </div>
  }
  else if (activeTab == "logs")
  {
    <div class="card shadow-sm p-3">
      <h5 class="mb-3">Recent Delivery Audit Logs</h5>
      @if (logs == null)
      {
        <p><em>Loading logs...</em></p>
      }
      else if (!logs.Any())
      {
        <p class="text-muted">No delivery logs recorded yet.</p>
      }
    </div>
  }

```

```

    }
    else
    {
        <div class="table-responsive">
            <table class="table table-striped table-sm align-middle">
                <thead>
                    <tr>
                        <th>Timestamp</th>
                        <th>Channel</th>
                        <th>Recipient</th>
                        <th>Articles</th>
                        <th>Status</th>
                        <th>Details</th>
                    </tr>
                </thead>
                <tbody>
                    @foreach (var log in logs)
                    {
                        <tr>
                            <td>@log.AttemptedAt.ToString("yyyy-MM-dd HH:mm:ss")</td>
                            <td><span class="badge bg-secondary">@log.ChannelType</span></td>
                            <td>@log.Recipient</td>
                            <td>@log.ArticleCount</td>
                            <td>
                                <span class="badge @(log.Status == "SUCCESS" ? "bg-success" : "bg-danger")">
                                    @log.Status
                                </span>
                            </td>
                            <td class="small text-muted">@(log.ErrorMessage ?? "None")</td>
                        </tr>
                    }
                </tbody>
            </table>
        </div>
    }
</div>

@code {
    private AdminMetrics? metrics;
    private List<Subscriber>? subscribers;
    private List<DeliveryLog>? logs;

    private string activeTab = "subscribers";
    private bool showDeleted = false;
    private bool isDispatching = false;
    private string? actionStatus;

    protected override async Task OnInitializedAsync()
    {
        await RefreshDataAsync();
    }

    private async Task SetTab(string tabName)
    {
        activeTab = tabName;
        await RefreshDataAsync();
    }

    private async Task RefreshDataAsync()
    {
        metrics = await AdminService.GetMetricsAsync();
        await LoadSubscribersAsync();
        logs = await AdminService.GetRecentDeliveryLogsAsync();
    }

    private async Task LoadSubscribersAsync()
    {
        subscribers = await AdminService.GetAllSubscribersAsync(showDeleted);
    }

    private async Task ToggleSoftDeleteAsync(int id)
    {
        await AdminService.ToggleSubscriberSoftDeleteAsync(id);
        await RefreshDataAsync();
    }

    private async Task DispatchSlackAsync()

```

```

    {
        isDispatching = true;
        actionStatus = null;

        int count = await DispatcherService.DispatchTopHeadlinesToSlackAsync(3);
        actionStatus = $"Manual dispatch complete. Sent notifications to {count} subscriber(s).";

        await RefreshDataAsync();
        isDispatching = false;
    }
}

```

7. Re-create Database (news.db)

Since DeliveryLogs is a new table, delete your existing news.db file in the project root and start the application so dbContext.Database.EnsureCreated() creates all 3 tables (NewsArticles, Subscribers, DeliveryLogs).

Navigate to <http://localhost:port/admin> to manage subscribers and audit logs.

User prompt: How can I restrict access to /admin using ASP.NET Core cookie authorization in Blazor?

Response: To restrict access to /admin in **Sonrisa** using ASP.NET Core cookie authentication in a Blazor Web App (.NET 8+), follow this step-by-step implementation.

Because Blazor Server uses WebSockets for interactive UI components, security sensitive actions—like issuing or destroying HTTP cookies—must be handled by standard **ASP.NET Core Minimal API / Controller endpoints** before redirecting back into the Blazor component pipeline.

Step 1: Configure Cookie Authentication in Program.cs

Update Program.cs to add authentication/authorization middleware and expose endpoint handlers for logging in and out.

```

using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.EntityFrameworkCore;
using Sonrisa.Data;
using Sonrisa.Services;

var builder = WebApplication.CreateBuilder(args);

// 1. Add Cookie Authentication & Authorization
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.Cookie.Name = "Sonrisa.AdminAuth";
        options.LoginPath = "/admin/login";
        options.AccessDeniedPath = "/admin/login";
        options.ExpireTimeSpan = TimeSpan.FromHours(8);
        options.SlidingExpiration = true;
    });

builder.Services.AddAuthorization();
builder.Services.AddCascadingAuthenticationState(); // Exposes AuthenticationState to Blazor components

// Existing DB and App Service registrations...
builder.Services.AddDbContextFactory<AppDbContext>(options =>
    options.UseSqlite("Data Source=news.db"));

builder.Services.AddHttpClient<NewsApiService>(client =>
{
    client.BaseAddress = new Uri("https://newsapi.org/");
    client.DefaultRequestHeaders.Add("User-Agent", "SonrisaNewsApp/1.0");
});
builder.Services.AddHttpClient<SlackNotificationService>();

builder.Services.AddTransient<SubscriberService>();
builder.Services.AddTransient<NotificationDispatcherService>();
builder.Services.AddTransient<AdminService>();

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

var app = builder.Build();

// Ensure DB schema is created
using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    dbContext.Database.EnsureCreated();
}

```

```

}

app.UseStaticFiles();
app.UseAntiforgery();

// 2. Enable Authentication & Authorization Middleware
app.UseAuthentication();
app.UseAuthorization();

// 3. HTTP Endpoints for Issuing and Revoking Auth Cookies
app.MapPost("/api/admin/login", async (HttpContext httpContext, string password, string? returnUrl) =>
{
    // Retrieve admin password from appsettings.json or configuration
    var configuredPassword = app.Configuration["Admin:Password"] ?? "SonrisaAdmin123!";

    if (password == configuredPassword)
    {
        var claims = new[]
        {
            new System.Security.Claims.Claim(System.Security.Claims.ClaimTypes.Name, "AdminUser"),
            new System.Security.Claims.Claim(System.Security.Claims.ClaimTypes.Role, "Admin")
        };

        var identity = new System.Security.Claims.ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
        var principal = new System.Security.Claims.ClaimsPrincipal(identity);

        await httpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, principal);

        return Results.Redirect(string.IsNullOrEmpty(returnUrl) ? "/admin" : returnUrl);
    }

    return Results.Redirect("/admin/login?error=invalid_credentials");
}).DisableAntiforgery(); // Simplified for internal form POST

app.MapGet("/api/admin/logout", async (HttpContext httpContext) =>
{
    await httpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
    return Results.Redirect("/admin/login");
});

app.MapRazorComponents<Sonrisa.Components.App>()
    .AddInteractiveServerRenderMode();

app.Run();

```

Step 2: Configure Admin Password in appsettings.json

Add an admin configuration block to appsettings.json:

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "NewsApi": {
    "ApiKey": "YOUR_NEWSAPI_ORG_KEY"
  },
  "Admin": {
    "Password": "SonrisaAdmin123!"
  }
}

```

Step 3: Wrap Router in Cascading Authentication (Components/App.razor)

Ensure your main App.razor file wraps the router view inside <CascadingAuthenticationState> so Blazor components can access the user context:

```

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <base href="/" />
  <link rel="stylesheet" href="bootstrap/bootstrap.min.css" />
  <link rel="stylesheet" href="app.css" />

```

```

    <link rel="stylesheet" href="Sonrisa.styles.css" />
    <HeadOutlet />
</head>

<body>
    <Routes />
    <script src="_framework/blazor.web.js"></script>
</body>

</html>

```

And update Components/Routes.razor to use AuthorizeRouteView:

```

<Router AppAssembly="typeof(Program).Assembly">
    <Found Context="routeData">
        <AuthorizeRouteView RouteData="routeData" DefaultLayout="typeof(Layout.MainLayout)">
            <NotAuthorized>
                <p class="p-4">You are not authorized to access this page. Redirecting...</p>
            </NotAuthorized>
        </AuthorizeRouteView>
        <FocusOnNavigate RouteData="routeData" Selector="h1" />
    </Found>
</Router>

```

Step 4: Create Login Page (Components/Pages/AdminLogin.razor)

Create Components/Pages/AdminLogin.razor with a standard HTML <form> that posts directly to our /api/admin/login HTTP endpoint:

```

@page "/admin/login"

<PageTitle>Sonrisa - Admin Login</PageTitle>

<div class="row justify-content-center mt-5">
    <div class="col-md-4">
        <div class="card shadow-sm">
            <div class="card-header bg-dark text-white text-center py-3">
                <h5 class="mb-0">Sonrisa Admin Access</h5>
            </div>
            <div class="card-body p-4">
                @if (IsError)
                {
                    <div class="alert alert-danger py-2">
                        Invalid admin password. Please try again.
                    </div>
                }

                <!-- Standard HTML form submitting to Minimal API endpoint to issue HTTP Auth Cookie -->
                <form action="/api/admin/login" method="post">
                    <div class="mb-3">
                        <label for="password" class="form-label">Admin Password</label>
                        <input type="password" id="password" name="password" class="form-control" placeholder="Enter" />
                    </div>
                    <button type="submit" class="btn btn-dark w-100">Log In</button>
                </form>
            </div>
        </div>
    </div>
</div>

@code {
    [SupplyParameterFromQuery]
    public string? Error { get; set; }

    private bool IsError => Error == "invalid_credentials";
}

```

Step 5: Secure the Admin Page (Components/Pages/Admin.razor)

Add @attribute [Authorize] at the top of Admin.razor to enforce authorization and add a Logout link:

```

@page "/admin"
@using Microsoft.AspNetCore.Authorization
@using Sonrisa.Models
@using Sonrisa.Services
@attribute [Authorize]
@Inject AdminService AdminService
@Inject NotificationDispatcherService DispatcherService
@rendermode InteractiveServer

<PageTitle>Sonrisa - Admin Dashboard</PageTitle>

```

```

<div class="container my-4">
  <div class="d-flex justify-content-between align-items-center mb-4">
    <div>
      <h2>Admin Dashboard</h2>
      <span class="badge bg-success">Authenticated Admin Session</span>
    </div>
    <div class="d-flex gap-2">
      <button class="btn btn-primary" @onclick="DispatchSlackAsync" disabled="@isDispatching">
        @(isDispatching ? "Dispatching..." : "Trigger Slack Dispatch Now")
      </button>
      <!-- Invokes cookie revocation endpoint -->
      <a href="/api/admin/logout" class="btn btn-outline-danger">Log Out</a>
    </div>
  </div>

  <!-- Rest of Admin Dashboard UI... -->
</div>

```

How It Works

1. Unauthenticated users navigating to /admin are intercepted by @attribute [Authorize] and redirected to /admin/login.
2. Entering the configured password submits a traditional POST request to /api/admin/login.
3. The server validates the credential and writes an encrypted Sonrisa.AdminAuth HTTP cookie to the client browser.
4. The client is redirected back to /admin, where the cookie is validated on every Blazor request.

User prompt: By visiting localhost:5128, I can only see: You are not authorized to access this page. Redirecting... I want the normal subscription page to be available without authorization, but the admin dashboard should only be visible after that.

Response: The issue occurs because AuthorizeRouteView in Components/Routes.razor was configured without specifying which routes actually require authorization, or because the default layout was set up to apply authorization rules globally.

When using @attribute [Authorize] on individual pages like Admin.razor, standard pages like Home.razor / Index.razor should remain publicly accessible.

Here is how to fix Routes.razor so the public subscription page loads freely while keeping /admin protected.

Step 1: Update Components/Routes.razor

Replace the contents of Components/Routes.razor with the following configuration. Using <NotAuthorized> inside AuthorizeRouteView handles unauthenticated requests specifically when hitting protected endpoints:

```

@using Microsoft.AspNetCore.Components.Authorization

<Router AppAssembly="typeof(Program).Assembly">
  <Found Context="routeData">
    <AuthorizeRouteView RouteData="routeData" DefaultLayout="typeof(Layout.MainLayout)">
      <NotAuthorized>
        @if (context.User.Identity?.IsAuthenticated != true)
        {
          <div class="container text-center my-5">
            <div class="card shadow-sm p-4 mx-auto" style="max-width: 450px;">
              <h4 class="mb-3 text-danger">Access Restricted</h4>
              <p class="text-muted mb-4">You must log in to view the admin area.</p>
              <a href="/admin/login" class="btn btn-dark w-100">Go to Admin Login</a>
            </div>
          </div>
        }
        else
        {
          <p class="p-4 text-center text-muted">You are not authorized to view this resource.</p>
        }
      </NotAuthorized>
    </AuthorizeRouteView>
    <FocusOnNavigate RouteData="routeData" Selector="h1" />
  </Found>
  <NotFound>
    <PageTitle>Not found</PageTitle>
    <LayoutView Layout="typeof(Layout.MainLayout)">
      <p role="alert" class="p-4">Sorry, there's nothing at this address.</p>
    </LayoutView>
  </NotFound>
</Router>

```

Step 2: Ensure Page Directives are Correct

Double-check the top directives on your main page (Components/Pages/Home.razor or Components/Pages/Index.razor) and Admin.razor:

Public Subscription Page (Components/Pages/Home.razor or Index.razor)

Ensure no `@attribute [Authorize]` directive exists at the top:

```
@page "/"
@using Sonrisa.Models
@using Sonrisa.Services
@inject SubscriberService SubscriberService
@rendermode InteractiveServer

<PageTitle>Sonrisa - News Headlines & Subscriptions</PageTitle>

<!-- Public Subscription UI -->
```

Admin Dashboard (Components/Pages/Admin.razor)

Keep `@attribute [Authorize]` at the top so only `/admin` requires authentication:

```
@page "/admin"
@using Microsoft.AspNetCore.Authorization
@attribute [Authorize]
@rendermode InteractiveServer

<PageTitle>Sonrisa - Admin Dashboard</PageTitle>

<!-- Admin UI -->
```

Admin Login Page (Components/Pages/AdminLogin.razor)

Ensure no authorization attribute is present here either:

```
@page "/admin/login"

<PageTitle>Sonrisa - Admin Login</PageTitle>

<!-- Login UI -->
```

How Route Processing Works Now

1. `http://localhost:5128/`: Evaluated as a public route. `AuthorizeRouteView` allows it through without checking credentials.
2. `http://localhost:5128/admin/login`: Evaluated as a public route. Loads the password form.
3. `http://localhost:5128/admin`: Evaluated against `@attribute [Authorize]`. If unauthenticated, `AuthorizeRouteView` triggers the `<NotAuthorized>` template and displays the login prompt option.

User prompt: When I try to login, I get this error: An unhandled exception occurred while processing the request. `BadRequestException: Required parameter "string password" was not provided from query string. lambda_method89(Closure , object , HttpContext) Stack Query Cookies Headers Routing BadRequestException: Required parameter "string password" was not provided from query string. lambda_method89(Closure , object , HttpContext) Microsoft.AspNetCore.Routing.EndpointMiddleware.Invoke(HttpContext httpContext) Microsoft.AspNetCore.Authorization.AuthorizationMiddleware.Invoke(HttpContext context) Microsoft.AspNetCore.Authentication.AuthenticationMiddleware.Invoke(HttpContext context) Microsoft.AspNetCore.Diagnostics.StatusCodePagesMiddleware.Invoke(HttpContext context) Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddlewareImpl.Invoke(HttpContext context)`

Response: The error occurs because standard Minimal API endpoint parameters default to reading from the **query string** unless specified otherwise. When your login form submits an `application/x-www-form-urlencoded` POST request, ASP.NET Core tries to locate password in the URL query string (`?password=...`), doesn't find it, and throws `BadRequestException`.

To fix this, mark the parameter with `[FromForm]` or accept the `IFormCollection` directly in your Minimal API endpoint in `Program.cs`.

Step 1: Update the Login Endpoint in Program.cs

Update your `app.MapPost("/api/admin/login", ...)` endpoint in `Program.cs` to explicitly bind parameters from the submitted form data:

```
using Microsoft.AspNetCore.Mvc; // Ensure this using statement is at the top of Program.cs

app.MapPost("/api/admin/login", async (
    HttpContext httpContext,
    [FromForm] string password,
    [FromForm] string? returnUrl) =>
{
    // Retrieve admin password from appsettings.json or configuration
    var configuredPassword = app.Configuration["Admin:Password"] ?? "SonrisaAdmin123!";
```

```

if (password == configuredPassword)
{
    var claims = new[]
    {
        new System.Security.Claims.Claim(System.Security.Claims.ClaimTypes.Name, "AdminUser"),
        new System.Security.Claims.Claim(System.Security.Claims.ClaimTypes.Role, "Admin")
    };

    var identity = new System.Security.Claims.ClaimsIdentity(claims, Microsoft.AspNetCore.Authentication.Cookies.CookieAuthenticationDefaults.AuthenticateProperties);
    var principal = new System.Security.Claims.ClaimsPrincipal(identity);

    await httpContext.SignInAsync(Microsoft.AspNetCore.Authentication.Cookies.CookieAuthenticationDefaults.AuthenticationScheme, principal);

    return Results.Redirect(string.IsNullOrEmpty(returnUrl) ? "/admin" : returnUrl);
}

return Results.Redirect("/admin/login?error=invalid_credentials");
}).DisableAntiforgery();

```

Alternative (Reading directly from IFormCollection)

If you prefer to parse form data without parameter binding attributes:

```

app.MapPost("/api/admin/login", async (HttpContext httpContext) =>
{
    var form = await httpContext.Request.ReadFormAsync();
    string password = form["password"].ToString();
    string? returnUrl = form["returnUrl"].ToString();

    var configuredPassword = app.Configuration["Admin:Password"] ?? "SonrisaAdmin123!";

    if (password == configuredPassword)
    {
        var claims = new[]
        {
            new System.Security.Claims.Claim(System.Security.Claims.ClaimTypes.Name, "AdminUser"),
            new System.Security.Claims.Claim(System.Security.Claims.ClaimTypes.Role, "Admin")
        };

        var identity = new System.Security.Claims.ClaimsIdentity(claims, Microsoft.AspNetCore.Authentication.Cookies.CookieAuthenticationDefaults.AuthenticateProperties);
        var principal = new System.Security.Claims.ClaimsPrincipal(identity);

        await httpContext.SignInAsync(Microsoft.AspNetCore.Authentication.Cookies.CookieAuthenticationDefaults.AuthenticationScheme, principal);

        return Results.Redirect(string.IsNullOrEmpty(returnUrl) ? "/admin" : returnUrl);
    }

    return Results.Redirect("/admin/login?error=invalid_credentials");
}).DisableAntiforgery();

```

Step 2: Ensure the HTML Form Attributes Match

In Components/Pages/AdminLogin.razor, verify that the <input> element name attribute matches the expected parameter:

```

<form action="/api/admin/login" method="post">
    <div class="mb-3">
        <label for="password" class="form-label">Admin Password</label>
        <input type="password" id="password" name="password" class="form-control" placeholder="Enter password..." re
    </div>
    <button type="submit" class="btn btn-dark w-100">Log In</button>
</form>

```

Restart the app, submit the form at /admin/login, and ASP.NET Core will bind the form field values correctly.

User prompt: This is my current navigation menu based on the default weather app in visual studio. Please update it to the current project: <div class="top-row ps-3 navbar navbar-dark"> <div class="container-fluid"> Sonrisa </div></div><input type="checkbox" title="Navigation menu" class="navbar-toggler" /><div class="nav-scrollable" onclick="document.querySelector('.navbar-toggler').click()"> <nav class="nav flex-column"> <div class="nav-item px-3"> <NavLink class="nav-link" href="" Match="NavLinkMatch.All"> Home </NavLink> </div> <div class="nav-item px-3"> <NavLink class="nav-link" href="counter"> Counter </NavLink> </div> <div class="nav-item px-3"> <NavLink class="nav-link" href="weather"> Weather </NavLink> </div> </nav></div>

Response: Here is the updated NavMenu.razor component tailored specifically for **Sonrisa**. It replaces the boilerplate "Counter" and "Weather" links with navigation for your public news dashboard, admin management area, and admin login/logout controls—complete with authentication state handling using <AuthorizeView>.

```

<div class="top-row ps-3 navbar navbar-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="">Sonrisa</a>
  </div>
</div>

<input type="checkbox" title="Navigation menu" class="navbar-toggler" />

<div class="nav-scrollable" onclick="document.querySelector('.navbar-toggler').click()">
  <nav class="nav flex-column">
    <!-- Public News & Subscription Home -->
    <div class="nav-item px-3">
      <NavLink class="nav-link" href="" Match="NavLinkMatch.All">
        <span class="bi bi-newspaper-nav-menu" aria-hidden="true"></span> Headlines & Subscribe
      </NavLink>
    </div>

    <!-- Protected Admin Dashboard -->
    <div class="nav-item px-3">
      <NavLink class="nav-link" href="admin">
        <span class="bi bi-speedometer2-nav-menu" aria-hidden="true"></span> Admin Dashboard
      </NavLink>
    </div>

    <div class="dropdown-divider my-2 border-secondary"></div>

    <!-- Dynamic Auth Links -->
    <AuthorizeView>
      <Authorized>
        <div class="nav-item px-3">
          <a class="nav-link text-warning" href="/api/admin/logout">
            <span class="bi bi-box-arrow-right-nav-menu" aria-hidden="true"></span> Admin Logout
          </a>
        </div>
      </Authorized>
      <NotAuthorized>
        <div class="nav-item px-3">
          <NavLink class="nav-link" href="admin/login">
            <span class="bi bi-shield-lock-fill-nav-menu" aria-hidden="true"></span> Admin Login
          </NavLink>
        </div>
      </NotAuthorized>
    </AuthorizeView>
  </nav>
</div>

```

Key Adjustments

1. Replaced Boilerplate Routes:

- /counter & /weather → Removed.
- / → Labeled as **Headlines & Subscribe** for public visitors.
- /admin → Added for managing subscribers and delivery logs.

2. Added AuthorizeView Integration:

- Shows an **Admin Logout** link (/api/admin/logout) when a user is signed in with an admin cookie.
- Shows an **Admin Login** link (admin/login) when the session is unauthenticated.

3. Icons & Accessibility:

- Uses Bootstrap icons appropriate for news (bi-newspaper), administration (bi-speedometer2), and authentication (bi-shield-lock-fill / bi-box-arrow-right).

User prompt: Similar to Slack notifications, I want to implement the email notifications to users based on whether they enabled email notifications or not.

Response: To add email dispatches to **Sonrisa**, we need an EmailNotificationService using standard .NET SMTP capabilities (SmtpClient), updated configuration settings for SMTP parameters, and integration into NotificationDispatcherService.

Step 1: Add Email Configuration to appsettings.json

Add SMTP server details to appsettings.json. For local development or testing, you can use services like Mailtrap, SendGrid SMTP, or a local SMTP capture tool (like Papercut or SMTP4Dev).

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "NewsApi": {
    "ApiKey": "YOUR_NEWSAPI_ORG_KEY"
  },
  "Admin": {
    "Password": "SonrisaAdmin123!"
  },
  "Smtp": {
    "Host": "smtp.mailtrap.io",
    "Port": 2525,
    "Username": "YOUR_SMTP_USERNAME",
    "Password": "YOUR_SMTP_PASSWORD",
    "FromAddress": "no-reply@sonrisa.com",
    "FromName": "Sonrisa News Digest",
    "EnableSsl": true
  }
}
```

Step 2: Create EmailNotificationService.cs (Services/EmailNotificationService.cs)

This service constructs HTML digest emails from top headlines and dispatches them via SMTP:

```
using System.Net;
using System.Net.Mail;
using System.Text;
using Sonrisa.Models;

namespace Sonrisa.Services;

public class EmailNotificationService
{
    private readonly IConfiguration _config;
    private readonly ILogger<EmailNotificationService> _logger;

    public EmailNotificationService(IConfiguration config, ILogger<EmailNotificationService> logger)
    {
        _config = config;
        _logger = logger;
    }

    public async Task<bool> SendNewsNotificationAsync(string recipientEmail, List<NewsArticle> articles)
    {
        var host = _config["Smtp:Host"];
        if (string.IsNullOrEmpty(host))
        {
            _logger.LogWarning("SMTP Host is not configured. Skipping email dispatch.");
            return false;
        }

        int port = int.TryParse(_config["Smtp:Port"], out var parsedPort) ? parsedPort : 587;
        var username = _config["Smtp:Username"];
        var password = _config["Smtp:Password"];
        var fromAddress = _config["Smtp:FromAddress"] ?? "no-reply@sonrisa.com";
        var fromName = _config["Smtp:FromName"] ?? "Sonrisa News Digest";
        bool enableSsl = bool.TryParse(_config["Smtp:EnableSsl"], out var ssl) && ssl;

        try
        {
            using var client = new SmtpClient(host, port)
            {
                EnableSsl = enableSsl
            };

            if (!string.IsNullOrEmpty(username) && !string.IsNullOrEmpty(password))
            {
                client.Credentials = new NetworkCredential(username, password);
            }

            var mailMessage = new MailMessage
            {
                From = new MailAddress(fromAddress, fromName),
                Subject = $"Sonrisa Daily Digest - Top {articles.Count} Headlines",
            }
        }
    }
}
```

```

        IsBodyHtml = true,
        Body = BuildEmailHtmlBody(articles)
    };

    mailMessage.To.Add(recipientEmail);

    await client.SendMailAsync(mailMessage);
    return true;
}
catch (Exception ex)
{
    _logger.LogError(ex, "Failed to send news email notification to {RecipientEmail}", recipientEmail);
    return false;
}
}

private static string BuildEmailHtmlBody(List<NewsArticle> articles)
{
    var sb = new StringBuilder();
    sb.Append("""
        <!DOCTYPE html>
        <html>
        <head>
            <style>
                body { font-family: Arial, sans-serif; background-color: #f8f9fa; color: #333; padding: 20px; }
                .container { max-width: 600px; margin: 0 auto; background: #ffffff; border-radius: 8px; padding:
                .header { text-align: center; border-bottom: 2px solid #0d6efd; padding-bottom: 10px; margin-bot
                .article { margin-bottom: 20px; padding-bottom: 15px; border-bottom: 1px solid #eee; }
                .article h3 { margin: 0 0 5px 0; font-size: 18px; }
                .article a { color: #0d6efd; text-decoration: none; }
                .article p { margin: 5px 0; color: #555; font-size: 14px; }
                .meta { font-size: 12px; color: #888; }
                .footer { text-align: center; font-size: 12px; color: #999; margin-top: 20px; }
            </style>
        </head>
        <body>
            <div class="container">
                <div class="header">
                    <h2>Sonrisa News Digest</h2>
                    <p>Your latest curated headlines</p>
                </div>
            """);

    foreach (var article in articles)
    {
        sb.AppendFormat("""
            <div class="article">
                <h3><a href="{0}" target="_blank">{1}</a></h3>
                <p>{2}</p>
                <div class="meta">Source: {3} | Published: {4}</div>
            </div>
            """,
            article.Url,
            WebUtility.HtmlEncode(article.Title),
            WebUtility.HtmlEncode(article.Description ?? "No description available."),
            WebUtility.HtmlEncode(article.Source ?? "Unknown"),
            article.PublishedAt?.ToString("MMM dd, yyyy HH:mm") ?? "N/A"
        );
    }

    sb.Append("""
        <div class="footer">
            <p>You are receiving this email because you subscribed to Sonrisa News updates.</p>
        </div>
    </body>
    </html>
    """);

    return sb.ToString();
}
}

```

Step 3: Register EmailNotificationService in Program.cs

Add the service registration to Program.cs:

```
builder.Services.AddTransient<EmailNotificationService>();
```

Step 4: Update NotificationDispatcherService.cs

Update NotificationDispatcherService.cs to inject EmailNotificationService and dispatch emails to subscribers who have NotifyViaEmail == true:

```
using Microsoft.EntityFrameworkCore;
using Sonrisa.Data;
using Sonrisa.Models;

namespace Sonrisa.Services;

public class NotificationDispatcherService
{
    private readonly IDbContextFactory<AppDbContext> _dbContextFactory;
    private readonly SlackNotificationService _slackService;
    private readonly EmailNotificationService _emailService;

    public NotificationDispatcherService(
        IDbContextFactory<AppDbContext> dbContextFactory,
        SlackNotificationService slackService,
        EmailNotificationService emailService)
    {
        _dbContextFactory = dbContextFactory;
        _slackService = slackService;
        _emailService = emailService;
    }

    /// <summary>
    /// Dispatches top headlines to all eligible active subscribers across Slack and Email channels.
    /// Returns total successful dispatches.
    /// </summary>
    public async Task<int> DispatchTopHeadlinesAsync(int topCount = 3)
    {
        using var db = await _dbContextFactory.CreateDbContextAsync();

        var recentArticles = await db.NewsArticles
            .OrderByDescending(a => a.PublishedAt ?? a.ImportedAt)
            .Take(topCount)
            .ToListAsync();

        if (!recentArticles.Any())
            return 0;

        int totalSuccesses = 0;

        // 1. Process Slack Subscriptions
        var slackSubscribers = await db.Subscribers
            .Where(s => s.NotifyViaSlack && !string.IsNullOrEmpty(s.SlackWebhookUrl))
            .ToListAsync();

        foreach (var subscriber in slackSubscribers)
        {
            bool sent = await _slackService.SendNewsNotificationAsync(subscriber.SlackWebhookUrl!, recentArticles);

            db.DeliveryLogs.Add(new DeliveryLog
            {
                ChannelType = "Slack",
                Recipient = subscriber.Email,
                ArticleCount = recentArticles.Count,
                Status = sent ? "SUCCESS" : "FAILED",
                ErrorMessage = sent ? null : "Failed to deliver payload to Slack webhook.",
                AttemptedAt = DateTime.UtcNow
            });

            if (sent) totalSuccesses++;
        }

        // 2. Process Email Subscriptions
        var emailSubscribers = await db.Subscribers
            .Where(s => s.NotifyViaEmail)
            .ToListAsync();

        foreach (var subscriber in emailSubscribers)
        {
            bool sent = await _emailService.SendNewsNotificationAsync(subscriber.Email, recentArticles);

            db.DeliveryLogs.Add(new DeliveryLog
            {
                ChannelType = "Email",
```

```

        Recipient = subscriber.Email,
        ArticleCount = recentArticles.Count,
        Status = sent ? "SUCCESS" : "FAILED",
        ErrorMessage = sent ? null : "Failed to send email via SMTP.",
        AttemptedAt = DateTime.UtcNow
    });

    if (sent) totalSuccesses++;
}

await db.SaveChangesAsync();
return totalSuccesses;
}
}

```

Step 5: Update Manual Dispatch in Admin.razor

In Components/Pages/Admin.razor, update the manual dispatch button handler to call the unified DispatchTopHeadlinesAsync method:

```

<button class="btn btn-primary" @onclick="DispatchNotificationsAsync" disabled="@isDispatching">
    @(isDispatching ? "Dispatching..." : "Trigger Dispatches Now (Slack & Email)")
</button>

@code {
    private async Task DispatchNotificationsAsync()
    {
        isDispatching = true;
        actionStatus = null;

        int count = await DispatcherService.DispatchTopHeadlinesAsync(3);
        actionStatus = $"Manual dispatch complete. Delivered {count} notification(s) across Slack and Email channels

        await RefreshDataAsync();
        isDispatching = false;
    }
}

```

User prompt: I want to use API tokens as defined in V2 for mailtrap.io, like in this example:

Response: To authenticate with Mailtrap API v2 using API tokens (Bearer token authentication in the HTTP Authorization header), you can update EmailNotificationService to use Mailtrap's REST API endpoint ([https://send.api.mailtrap.io/api/send] (https://send.api.mailtrap.io/api/send)) via HttpClient instead of traditional SMTP.

Step 1: Update appsettings.json

Add your Mailtrap v2 API token and sender configuration:

```

{
  "Mailtrap": {
    "ApiToken": "YOUR_MAILTRAP_V2_API_TOKEN",
    "FromEmail": "no-reply@yourdomain.com",
    "FromName": "Sonrisa News Digest"
  }
}

```

Step 2: Implement Mailtrap API v2 Payload Models

Create a lightweight model matching the Mailtrap v2 REST API contract:

```

using System.Text.Json.Serialization;

namespace Sonrisa.Models.Mailtrap;

public class MailtrapSendRequest
{
    [JsonPropertyName("from")]
    public MailtrapContact From { get; set; } = new();

    [JsonPropertyName("to")]
    public List<MailtrapContact> To { get; set; } = new();

    [JsonPropertyName("subject")]
    public string Subject { get; set; } = string.Empty;

    [JsonPropertyName("html")]
    public string Html { get; set; } = string.Empty;

    [JsonPropertyName("category")]

```

```

    public string Category { get; set; } = "News Digest";
}

public class MailtrapContact
{
    [JsonPropertyName("email")]
    public string Email { get; set; } = string.Empty;

    [JsonPropertyName("name")]
    public string? Name { get; set; }
}

```

Step 3: Update EmailNotificationService.cs

Rewrite EmailNotificationService.cs to issue HTTP POST requests with the Bearer API token:

```

using System.Net;
using System.Net.Http.Headers;
using System.Net.Http.Json;
using System.Text;
using Sonrisa.Models;
using Sonrisa.Models.Mailtrap;

namespace Sonrisa.Services;

public class EmailNotificationService
{
    private readonly HttpClient _httpClient;
    private readonly IConfiguration _config;
    private readonly ILogger<EmailNotificationService> _logger;

    public EmailNotificationService(HttpClient httpClient, IConfiguration config, ILogger<EmailNotificationService>
    {
        _httpClient = httpClient;
        _config = config;
        _logger = logger;
    }

    public async Task<bool> SendNewsNotificationAsync(string recipientEmail, List<NewsArticle> articles)
    {
        var apiToken = _config["Mailtrap:ApiToken"];
        var fromEmail = _config["Mailtrap:FromEmail"] ?? "no-reply@sonrisa.com";
        var fromName = _config["Mailtrap:FromName"] ?? "Sonrisa News Digest";

        if (string.IsNullOrEmpty(apiToken))
        {
            _logger.LogWarning("Mailtrap API token is not configured. Skipping email dispatch.");
            return false;
        }

        var payload = new MailtrapSendRequest
        {
            From = new MailtrapContact { Email = fromEmail, Name = fromName },
            To = new List<MailtrapContact> { new() { Email = recipientEmail } },
            Subject = $"Sonrisa Daily Digest - Top {articles.Count} Headlines",
            Html = BuildEmailHtmlBody(articles)
        };

        try
        {
            using var request = new HttpRequestMessage(HttpMethod.Post, "https://send.api.mailtrap.io/api/send");

            // Set Bearer token authorization header as required by Mailtrap API v2
            request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", apiToken);
            request.Content = JsonContent.Create(payload);

            var response = await _httpClient.SendAsync(request);

            if (response.IsSuccessStatusCode)
            {
                return true;
            }

            var responseBody = await response.Content.ReadAsStringAsync();
            _logger.LogError("Mailtrap API v2 request failed with status {StatusCode}: {ResponseBody}", response.Sta
            return false;
        }
        catch (Exception ex)
        {

```

```

        _logger.LogError(ex, "Failed to send email via Mailtrap API v2 to {RecipientEmail}", recipientEmail);
        return false;
    }
}

private static string BuildEmailHtmlBody(List<NewsArticle> articles)
{
    var sb = new StringBuilder();
    sb.Append("""
        <!DOCTYPE html>
        <html>
        <head>
            <style>
                body { font-family: Arial, sans-serif; background-color: #f8f9fa; color: #333; padding: 20px; }
                .container { max-width: 600px; margin: 0 auto; background: #ffffff; border-radius: 8px; padding: 10px; }
                .header { text-align: center; border-bottom: 2px solid #0d6efd; padding-bottom: 10px; margin-bottom: 10px; }
                .article { margin-bottom: 20px; padding-bottom: 15px; border-bottom: 1px solid #eee; }
                .article h3 { margin: 0 0 5px 0; font-size: 18px; }
                .article a { color: #0d6efd; text-decoration: none; }
                .article p { margin: 5px 0; color: #555; font-size: 14px; }
                .meta { font-size: 12px; color: #888; }
                .footer { text-align: center; font-size: 12px; color: #999; margin-top: 20px; }
            </style>
        </head>
        <body>
            <div class="container">
                <div class="header">
                    <h2>Sonrisa News Digest</h2>
                    <p>Your latest curated headlines</p>
                </div>
            </div>
            <div class="article">
                <h3><a href="{0}" target="_blank">{1}</a></h3>
                <p>{2}</p>
                <div class="meta">Source: {3} | Published: {4}</div>
            </div>
            """,
            article.Url,
            WebUtility.HtmlEncode(article.Title),
            WebUtility.HtmlEncode(article.Description ?? "No description available."),
            WebUtility.HtmlEncode(article.Source ?? "Unknown"),
            article.PublishedAt?.ToString("MMM dd, yyyy HH:mm") ?? "N/A"
        );
    }

    sb.Append("""
        <div class="footer">
            <p>You are receiving this email because you subscribed to Sonrisa News updates.</p>
        </div>
    </body>
    </html>
    """);

    return sb.ToString();
}
}

```

Step 4: Register HttpClient with Typed Service in Program.cs

In Program.cs, replace the transient registration with a typed HttpClient registration:

```
builder.Services.AddHttpClient<EmailNotificationService>();
```

User prompt: How can I make the /news page the homepage?

Response: To make the news subscription page serve as the homepage (/), update the page routing directive in your Blazor component.

1. Update Route Directives in the News Component

Locate your main news component (e.g., Components/Pages/News.razor or Components/Pages/Home.razor) and ensure its @page directive includes "/":

```
@page "/"
@page "/news"
@using Sonrisa.Models
@using Sonrisa.Services
@inject SubscriberService SubscriberService
@rendermode InteractiveServer
```

```
<PageTitle>Sonrisa - News Headlines & Subscriptions</PageTitle>
```

```
<!-- News page content here -->
```

Adding `@page "/"` tells the Blazor router to render this component when users visit the root URL (<http://localhost:5128/>). Keeping `@page "/news"` ensures any existing links or bookmarks pointing to `/news` continue to work.

2. (Optional) Remove or Rename Boilerplate `Index.razor`

If you have a leftover default `Components/Pages/Index.razor` or `Home.razor` file that still has `@page "/"`, either:

- **Option A:** Remove `@page "/"` from that file so it doesn't conflict with your news component, or
- **Option B:** Delete the unused component file entirely.

3. Update `NavMenu.razor`

In `Components/Layout/NavMenu.razor`, update the brand link and main home navigation link so both point to `/`:

```
<div class="top-row ps-3 navbar navbar-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="/">Sonrisa</a>
  </div>
</div>

<!-- Nav item -->
<div class="nav-item px-3">
  <NavLink class="nav-link" href="/" Match="NavLinkMatch.All">
    <span class="bi bi-newspaper-nav-menu" aria-hidden="true"></span> Headlines & Subscribe
  </NavLink>
</div>
```

Now visiting <http://localhost:5128/> will load your news headlines and subscription controls.

User prompt: Mailtrap.io won't allow me to send emails from localhost with this solution. Help me solve that.

Response: Mailtrap's REST API ([send.api.mailtrap.io](https://api.mailtrap.io)) rejects sending requests when using unverified domain addresses (like `@localhost` or `@sonrisa.local`) in the `from` field. When testing on localhost, Mailtrap requires specific setup depending on whether you are using **Mailtrap Email Sending** or **Mailtrap Email Testing (Inboxes)**.

Here is how to resolve this depending on which Mailtrap service you are using.

Solution 1: Use Mailtrap Email Testing (Inbox / Sandbox) — Recommended for Local Dev

If you want a safe sandbox where emails are caught in a fake inbox without delivering real messages, use the **Email Testing API**.

1. Endpoint and URL

- **Endpoint:** `[https://sandbox.api.mailtrap.io/api/send/](https://sandbox.api.mailtrap.io/api/send/){inbox_id}`
- Find your `{inbox_id}` in Mailtrap: **Email Testing** → **Inboxes** → Select Inbox → Look at the URL or the **API** integration tab.

2. Update `appsettings.json`

```
{
  "Mailtrap": {
    "ApiToken": "YOUR_INBOX_API_TOKEN",
    "InboxId": "1234567",
    "FromEmail": "any-sender@example.com",
    "FromName": "Sonrisa News Digest"
  }
}
```

3. Update `EmailNotificationService.cs`

Update the endpoint URL in `EmailNotificationService.cs`:

```
var inboxId = _config["Mailtrap:InboxId"];
var endpoint = $"https://sandbox.api.mailtrap.io/api/send/{inboxId}";

using var request = new HttpRequestMessage(HttpMethod.Post, endpoint);
```

```
request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", apiToken);  
request.Content = JsonContent.Create(payload);
```

(In sandbox mode, FromEmail can be any arbitrary email address like test@example.com.)

Solution 2: Use Mailtrap Email Sending (Production API) with mailtrap.site Domain

If you are using **Email Sending** (send.api.mailtrap.io) instead of Email Testing, Mailtrap strictly enforces domain verification.

To test from localhost using the Sending API without verifying your own custom domain:

1. Log in to Mailtrap and navigate to **Email Sending** → **Sending Domains**.
2. Mailtrap provides a built-in demo domain ending in @mailtrap.site (e.g., demomailtrap.com or test@mailtrap.site).
3. Set your FromEmail in appsettings.json to match that exact address:

```
{  
  "Mailtrap": {  
    "ApiToken": "YOUR_SENDING_API_TOKEN",  
    "FromEmail": "pass-your-demo-email@demomailtrap.com",  
    "FromName": "Sonrisa News Digest"  
  }  
}
```

Note: Mailtrap's demo domain limits delivery strictly to the email address registered with your Mailtrap account.

Solution 3: Standard SMTP Client (Zero API Payload Restrictions)

If you prefer to bypass API payload constraints entirely on localhost, switch back to Mailtrap's SMTP credentials under **Email Testing** → **Inboxes** → **SMTP Settings**:

```
{  
  "Smtp": {  
    "Host": "sandbox.smtp.mailtrap.io",  
    "Port": 2525,  
    "Username": "YOUR_SMTP_USERNAME",  
    "Password": "YOUR_SMTP_PASSWORD",  
    "FromAddress": "test@sonrisa.local",  
    "FromName": "Sonrisa News Digest"  
  }  
}
```

Using standard SMTP allows any dummy FromAddress and catches all outgoing local test emails directly inside your Mailtrap sandbox inbox.
